

DEEPSHIELD AES - A DYNAMIC S-BOX BASED ENHANCEMENT FOR SECURE FILE ENCRYPTION

*A Project Report submitted
in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

K. Susmitha Lakshmi	22B01A0579
K. Sree Karthika	22B01A05A0
K. Harika	22B01A05A2
M. Varshini	22B01A05B6
M. Valli Gayathri	22B01A05C1

Under the esteemed guidance of

Dr. P. Sri Ram Chandra M.Tech, Ph.D
Associate Professor



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SHRI VISHNU ENGINEERING COLLEGE FOR WOMEN(A)**
(Approved by AICTE, Accredited by NBA & NAAC, Affiliated to JNTU Kakinada)
BHIMAVARAM – 534 202
2025 – 2026

SHRI VISHNU ENGINEERING COLLEGE FOR WOMEN(A)

(Approved by AICTE, Accredited by NBA & NAAC, Affiliated to JNTU Kakinada)

BHIMAVARAM – 534 202

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

*This is to certify that the project entitled "**DeepShield AES - A Dynamic S-Box Based Enhancement for Secure File Encryption**", is being submitted by **K. Susmitha Lakshmi, K. Sree Karthika, K. Harika, M. Varshini, M. Valli Gayatri** bearing the **Regd. No's 22B01A0579, 22B01A05A0, 22B01A05A2, 22B01A05B6, 22B01A05C1** in partial fulfillment of the requirements for the award of the degree of "**Bachelor of Technology in Computer Science & Engineering**" is a record of Bonafide work carried out by her under my guidance and supervision during the academic year 2025-2026 and it has been found worthy of acceptance according to the requirements of the university.*

Internal Guide

Head of the Department

External Examiner

ACKNOWLEDGEMENTS

The successful completion of this project would not have been possible without the support, guidance, and encouragement of several individuals and organizations. We take this opportunity to express our sincere gratitude to all those who contributed to the accomplishment of this work.

We express our profound gratitude to **Sri K. V. Vishnu Raju, Chairman of SVES**, for their constant support and encouragement toward all our academic endeavors. We are highly thankful to **Dr. P. Srinivasa Raju, Director of Student Affairs, SVES, Bhimavaram**, for their continuous motivation and support.

We would like to extend our sincere thanks to **Dr. G. Srinivasa Rao, Principal of SVECW** and **Mr. P. Venkata Raju, Vice Principal of SVECW**, for their constant insights and encouragement.

We place our deep sense of gratitude to **Dr. P. Kiran Sree, Head of the Department of Computer Science and Engineering**, for their valuable suggestions and guidance, which helped us complete this project within the stipulated time.

Finally, we express our heartfelt gratitude to our **project guide, Dr. P. Sri Ram Chandra, Associate Professor**, for their continuous support, expert guidance, and valuable insights throughout the project. Their dedication and mentorship was greatly contributed to the successful completion of this project.

We are deeply thankful to our **Project Coordinator, Mr. Nagaraju, Professor**, for their indispensable guidance and unwavering support throughout the project. We would also like to express our sincere thanks to our **PRC members, Dr. A. Seenu and Dr. M. Narasimha Raju**, for their valuable suggestions and constructive feedback throughout this project.

Project Associates:

K. Susmitha Lakshmi	22B01A0579
K. Sree Karthika	22B01A05A0
K. Harika	22B01A05A2
M. Varshini	22B01A05B6
M. Valli Gayathri	22B01A05C1

ABSTRACT

In today's digital world, a large amount of sensitive information such as personal documents, financial data, and confidential files is stored in digital form. With the increasing use of cloud storage systems and online platforms, ensuring the security and privacy of stored data has become a critical concern. Unauthorized access, data leakage, and cyber-attacks pose serious threats to digital information, making secure data storage systems an essential requirement.

Traditional file storage systems often rely on basic security mechanisms that may not provide sufficient protection against modern cyber threats. Therefore, advanced encryption techniques are required to ensure that stored files remain confidential and protected from unauthorized users. Encryption converts readable data into an unreadable format, which can only be accessed by authorized users possessing the correct decryption key.

By integrating "DeepShield AES – A Dynamic S-Box Based Enhancement for Secure File Encryption", a secure system is designed to provide a platform for storing and managing files using strong cryptographic techniques. The system implements AES-GCM encryption along with a Chaotic S-Box mechanism to enhance data security and improve resistance against cryptographic attacks. This approach ensures that the stored data remains protected even if unauthorized access to the storage system occurs.

The proposed system provides functionalities such as secure file upload, encrypted storage, and controlled file retrieval. Users can safely store their files within the system, and the encryption mechanism ensures that the files cannot be accessed without proper authentication and decryption. The integration of encryption techniques with a modern web-based application allows the system to be used easily while maintaining a high level of security.

The application is developed using modern web technologies such as ReactJS for the user interface and MongoDB for database management. These technologies provide a scalable and efficient platform for handling user data and managing file storage operations. By combining advanced cryptographic techniques with modern web development frameworks, the proposed system offers a secure and efficient solution for digital file storage.

TABLE OF CONTENTS

S.No.	Topic Name	Page No.
1	INTRODUCTION	1 – 5
	1.1 Background of Digital Data Security	
	1.2 Overview of AES Encryption	
	1.3 Problem Statement	
	1.4 Motivation	
	1.5 Objectives and Scope	
2	SYSTEM ANALYSIS	6 – 10
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Feasibility Study	
3	SYSTEM REQUIREMENTS SPECIFICATION	11 – 15
	3.1 Software Requirements	
	3.2 Hardware Requirements	
4	SYSTEM DESIGN	16 – 30
	4.1 Introduction	
	4.2 System Architecture	
	4.2.1 Algorithm Architecture	
	4.2.1.1 File Upload and Encryption Process	
	4.2.1.2 File Retrieval and Decryption Process	
	4.2.2 Application Architecture	
	4.3 UML Diagrams	
	4.4 Database Design	
5	SYSTEM IMPLEMENTATION	31 – 45
	5.1 Introduction	
	5.2 Project Modules	
	5.3 Algorithms	
	5.4 Screens	
	5.5 System Deployment	
6	SYSTEM TESTING	46 – 51
	6.1 Introduction	
	6.2 Testing Modules	
	6.3 Test Cases	
7	RESULTS OF PROPOSED SYSTEM	52-53

8	CONCLUSION	54 – 55
9	BIBLIOGRAPHY	56-57
10	APPENDIX	58-61
	10.1 Appendix-A	
	10.2 Appendix-B	
	10.2.1 Introduction to Python	
	10.2.2 Introduction to Flask	
	10.2.3 Introduction to HTML	
	10.2.4 Introduction to Tailwind CSS	
	10.5.5 Introduction to JavaScript	

LIST OF FIGURES

S.No.	Figure	Page No.
1	4.2.1 Algorithm Architecture	17
2	4.2.2 Application Architecture	22
3	4.3.1 Use Case Diagram	26
4	4.3.2 Sequence Diagram	28
5	4.3.3 Database Design	29
6	5.4.1 Home Page	38
7	5.4.2 SignUp Page	39
8	5.4.3 Login Page	39
9	5.4.4 OTP Verification Page	40
10	5.4.5 Dashboard	40
11	5.4.6 Encrypt Page	41
12	5.4.7 Decrypt Page	42
13	5.4.8 Activity Page	42
14	5.4.9 Instructions Page	43
15	5.4.10 Help Page	43
16	6.3.1 Successful Encryption	48
17	6.3.2 Encryption with Missing Data	49
18	6.3.3 Successful File Decryption	50
19	6.3.4 Decryption with Incorrect Password	50
20	6.3.5 Encryption of Same File with Same Password Multiple Times	51
21	10.1.1 QR Code	58
22	10.1.2 QR Code	59

1. INTRODUCTION

1.1 Background of Digital Data Security

In the modern digital era, the amount of information generated, shared and stored electronically has increased significantly. Individuals and organizations rely heavily on digital platforms to store important information such as personal documents, financial records, academic data and confidential business files. With the rapid growth of cloud computing, web applications and online storage platforms, managing digital information has become easier and more efficient. However, the increased use of digital storage systems has also introduced serious security challenges. Cyber-attacks, unauthorized access and data breaches have become common threats that can compromise sensitive information. Therefore, ensuring the confidentiality, integrity and security of stored data has become a major concern in today's computing environment.

Traditional storage systems often rely on basic authentication methods such as usernames and passwords. Although these methods provide a basic level of protection, they are not always sufficient to prevent advanced cyber threats. Attackers can exploit system vulnerabilities or gain unauthorized access to stored files, which may lead to data theft or misuse. When sensitive data is stored without proper protection, it becomes highly vulnerable to security risks. To overcome these challenges, stronger security mechanisms such as encryption techniques are required to protect digital information.

1.2 Overview of AES encryption

Encryption is a fundamental technique used in information security to protect sensitive data from unauthorized access. It is the process of converting readable information, known as plaintext into an unreadable format called ciphertext using mathematical algorithms and cryptographic keys. The purpose of encryption is to ensure that even if data is intercepted or accessed by unauthorized individuals, it cannot be understood without the appropriate decryption key. Only users who possess the correct key can convert the encrypted ciphertext back into its original readable form through the decryption process. In modern digital systems, encryption plays a crucial role in maintaining the confidentiality, integrity, and privacy of data during storage and transmission.

Encryption techniques are widely used in various applications such as secure communication, online banking, cloud storage, email security and protection of personal and organizational data. When information is transmitted over networks it may pass through multiple systems and networks where it could potentially be intercepted. Encryption helps to prevent attackers from understanding the captured data, thereby safeguarding sensitive information from misuse. In addition to confidentiality, many modern encryption methods also provide mechanisms for verifying data integrity and authenticity, ensuring that the information has not been altered during transmission or storage.

Cryptographic algorithms used for encryption are generally classified into two main categories: symmetric encryption and asymmetric encryption. In symmetric encryption, the same key is used for both encryption and decryption processes. This method is efficient and widely used for encrypting large amounts of data. Asymmetric encryption, on the other hand, uses two different keys: a public key for encryption and a private key for decryption. While asymmetric encryption is useful for secure key exchange and authentication, symmetric encryption algorithms are generally faster and more suitable for bulk data encryption.

Among the various symmetric encryption algorithms available today, the Advanced Encryption Standard (AES) is one of the most widely adopted and trusted cryptographic standards. AES was established by the National Institute of Standards and Technology (NIST) in 2001 as a replacement for the older Data Encryption Standard (DES). AES is based on a substitution-permutation network and operates on fixed-size data blocks of 128 bits while supporting key sizes of 128, 192, and 256 bits. The algorithm performs several rounds of transformation operations such as substitution, permutation and mixing of data to produce highly secure ciphertext.

AES is widely used in many modern security applications due to its strong resistance to known cryptographic attacks, efficient performance, and flexibility in different computing environments. It is commonly used in secure communication protocols, file encryption systems, wireless security standards and cloud storage services. Because of its reliability and strong security properties, AES has become the global standard for protecting sensitive digital information.

1.3 Problem Statement

Several modified versions of the Advanced Encryption Standard (AES) encryption algorithm have been proposed over the years to enhance its security and performance. These improvements

are particularly important for modern computing environments such as Internet of Things (IoT) devices, embedded systems, and other resource-constrained platforms where memory, processing power, and energy consumption are limited. Researchers have attempted to modify different components of the AES algorithm, such as the key generation process, substitution boxes (S-Boxes), and encryption rounds, in order to increase randomness, strengthen diffusion, and improve resistance against cryptographic attacks.

However, many of these modifications also introduce certain limitations. Some approaches reduce the avalanche effect, which is an important property of cryptographic algorithms where a small change in input produces a significant change in output. Other techniques may create dependencies on specific input data patterns or reduce the level of confusion and diffusion when certain steps of the algorithm are simplified or skipped. Additionally, some enhanced AES models increase implementation complexity by introducing additional operations, logic circuits, or computational overhead. This can make the algorithm less suitable for low-power devices and systems with limited computational resources.

These challenges highlight the need for a balanced approach when improving AES-based encryption techniques. An ideal solution should enhance the security strength of the algorithm by providing high randomness, better diffusion, and strong resistance against cryptographic attacks, while still maintaining a simple and efficient implementation. Such an approach would allow the encryption system to operate effectively even on constrained devices without significantly affecting processing speed, energy consumption, or system reliability.

1.4 Motivation

The limitations identified in existing AES approaches highlight the importance of designing an encryption system that can improve security without increasing implementation complexity or reducing important cryptographic properties such as the avalanche effect, confusion and diffusion. In many modern applications, particularly those involving cloud storage, IoT devices and web-based platforms, it is essential to protect sensitive data while ensuring that the encryption process remains efficient and reliable. Therefore, there is a strong motivation to develop encryption solutions that provide enhanced security while maintaining a balance between computational efficiency, implementation simplicity, and robustness against potential attacks.

The proposed project, "DeepShield AES: A Dynamic S-Box based Enhancement for Secure File Encryption" aims to provide a secure web application for storing and managing text based digital files. The system focuses on protecting files before they are being stored, ensuring that sensitive information remains confidential. In this project, encryption is implemented using AES

in Galois Counter Mode (AES-GCM), which is known for providing both data confidentiality and authentication. AES GCM not only encrypts the data but also generates an authentication tag that verifies the integrity of the encrypted information. This ensures that any unauthorized modification or tampering of the stored data can be detected, thereby improving the overall security of the system.

To further strengthen the encryption process, the proposed system incorporates a Chaotic S Box mechanism. The S-Box is an important component of encryption algorithms that introduces non-linearity into the transformation process. Traditional S-Boxes are usually fixed and may have certain limitations against advanced cryptographic attacks. In this project, a chaos-based S-Box is used to enhance the security of the encryption process. Chaotic systems possess properties such as randomness, sensitivity to initial conditions and unpredictability. By integrating chaotic behavior into S-Box design, the encryption algorithm becomes more complex and resistant to attacks such as differential and linear cryptanalysis. The combination of AES-GCM encryption with a Chaotic S-Box increases the overall strength of the cryptographic system and provides better protection for sensitive data.

The DeepShield AES system allows users to securely upload, store and retrieve files through a web-based application. When a file is uploaded, it is encrypted using the proposed encryption mechanism before being stored in the database. This ensures that even if unauthorized users gain access to the storage system, the files remain protected in encrypted form. Only authorized users with the correct credentials and decryption process can access the original files.

The application is developed using modern web technologies to provide an efficient and user-friendly system. The frontend of the application is developed using ReactJS, which enables the creation of a responsive and interactive user interface. ReactJS helps manage user interactions efficiently and provides a smooth experience while using the system. The backend data management is supported by MongoDB, a NoSQL database that allows flexible and scalable storage of user data and file information. MongoDB is capable of handling large volumes of data and is widely used in modern web applications.

1.5 Objectives and Scope

The primary objective of the DeepShield AES system is to design a secure and high-performance encryption framework for protecting sensitive data. It enhances the traditional AES

algorithm by integrating a chaotic S-box, which introduces greater randomness and unpredictability, thereby strengthening resistance against cryptographic attacks. In addition, the use of AES in Galois/Counter Mode (GCM) ensures faster encryption and built-in data integrity verification. The system also focuses on evaluating its effectiveness by comparing it with standard AES using key metrics such as avalanche effect, SAC, BIC, nonlinearity, and differential uniformity, ensuring both theoretical strength and practical reliability.

The scope of the system extends to real-world applications where data security and efficiency are critical, such as large organizations, cloud storage systems and secure communication platforms. By combining advanced encryption techniques with a full-stack implementation (Flask, Node.js, and React), DeepShield provides a scalable and adaptable solution for secure file handling. It not only ensures confidentiality and integrity of data but also lays the foundation for future enhancements like AI-based threat detection and enterprise-level security systems, making it suitable for modern cybersecurity demands.

2. SYSTEM ANALYSIS

2.1 Existing System

Advanced Encryption Standard (AES) is one of the most widely used symmetric encryption algorithms for securing digital data. AES and its authenticated mode AES-GCM provide strong confidentiality and integrity for data transmission in modern communication systems. However, several studies have identified limitations related to the key schedule, static substitution box (S-box) and computational complexity. Many existing approaches attempt to improve AES by modifying key generation, round operations or S-box construction. Despite these improvements, challenges remain in achieving stronger resistance against cryptanalysis while maintaining efficiency for large-scale data encryption.

Features of Existing System

- Standard AES Encryption – Uses the AES algorithm to encrypt data securely with fixed cryptographic operations such as SubBytes, ShiftRows, MixColumns, and AddRoundKey.
- Authenticated Encryption (AES-GCM) – Provides both confidentiality and integrity by combining encryption with authentication mechanisms.
- Static S-Box Structure – Utilizes a fixed substitution box for nonlinear transformation during encryption.
- Key Schedule Mechanism – Generates round keys from the original encryption key for each encryption round.
- High-Speed Encryption – AES-GCM is optimized for high-speed data communication and is widely used in secure network protocols.
- Lightweight Variants for Embedded Systems – Some AES modifications are designed to improve performance in IoT devices and embedded environments.

Drawbacks of Existing System

- Dependence on Static S-Box – The fixed S-box structure may become vulnerable to algebraic, linear, and differential cryptanalysis.
- Key Schedule Weakness – Partial leakage of sub-keys may allow attackers to reconstruct remaining round keys.
- High Computational Overhead – Some modified AES versions increase execution time due to additional operations.

- Limited Adaptability – Static cryptographic components reduce flexibility against evolving cryptographic attacks.
- Incomplete Security Evaluation – Some proposed AES variants are tested only on small datasets and lack extensive cryptanalysis.
- Challenges in Chaos-Based Methods – Chaotic S-box approaches may suffer from finite precision issues and difficulties in secure transmission.

2.2 Proposed System

- Dynamic Chaotic S-Box Generation – Generates a key-dependent chaotic substitution box using nonlinear chaotic maps, ensuring higher randomness and stronger resistance against cryptanalysis.
- Secure Key Derivation Mechanism – Uses PBKDF2 with SHA-512 and a random salt to derive strong cryptographic keys from user passwords, preventing brute-force and dictionary attacks.
- Chunk-Wise File Processing – Splits large files into smaller chunks during encryption and decryption, enabling efficient handling of large-scale text data while reducing memory consumption.
- Authenticated Encryption – Ensures both data confidentiality and integrity by verifying authentication tags during decryption, preventing tampering of encrypted files.
- Key-Dependent Security Layer – Generates a unique S-Box for each encryption session based on the derived key, introducing additional confusion and making attacks significantly harder.
- Efficient Large-File Encryption – Designed to encrypt large files securely while maintaining competitive performance suitable for real-time applications such as secure cloud storage and data transmission.

Objectives:

- Dynamic Chaotic S-Box Generation
- Secure Authenticated File Encryption with Integrity Verification
- Strong Key Derivation with PBKDF2-SHA512
- Efficient chunk-wise processing of large files
- Resistance Against Cryptographic Attacks:
 - Brute force attacks

- Dictionary attacks
- Rainbow table attacks
- Known plaintext attacks
- Differential cryptanalysis
- Data tampering attempts

2.3 Feasibility Study:

A feasibility study is an important step in determining whether a proposed system can be successfully developed and implemented. It evaluates several aspects such as technical capability, economic viability, legal considerations and user acceptance before allocating significant resources to the project. By carefully analyzing these factors, developers and stakeholders can understand the practicality, benefits, and potential limitations of the system.

In the case of the proposed encryption system, the feasibility study focuses on evaluating the practicality of implementing an enhanced AES-GCM encryption scheme integrated with a dynamic chaotic S-Box and a secure key derivation mechanism. The study also analyzes the system's capability to securely process and encrypt large volumes of data while maintaining high levels of security and efficiency.

Through technical, behavioral and economic analysis, the feasibility study ensures that the proposed encryption model is both practical and beneficial for secure data storage and transmission. This evaluation also helps in identifying potential risks, limitations, and improvements required for successful implementation. Overall, the feasibility study acts as a guide for understanding whether the proposed system can be implemented effectively using the available technology, resources, and infrastructure.

Technical Feasibility:

The proposed system is technically feasible as it utilizes well-established cryptographic algorithms and modern programming tools. The encryption model integrates **AES-256 in Galois/Counter Mode (AES-GCM)** with a **key-dependent chaotic S-Box**, which enhances the overall security and provides strong resistance against different types of cryptographic attacks.

The system is implemented using Python along with reliable cryptographic libraries that support secure encryption, authentication and key generation operations. Additionally, the system incorporates **PBKDF2 with SHA-512** for password-based key derivation, which significantly strengthens the encryption process and protects against brute-force or dictionary attacks.

Another important technical feature of the system is the **chunk-wise file encryption mechanism**, which allows large files to be divided into smaller segments during encryption and decryption. This improves system efficiency by reducing memory usage and enabling the encryption of large files without performance degradation. As a result, the system is scalable and capable of handling real-world data encryption requirements effectively.

Behavioral Feasibility:

Behavioral feasibility focuses on how easily users can adapt to and interact with the proposed system. The encryption system is designed with a simple and user-friendly workflow that allows users to encrypt and decrypt files securely without requiring advanced knowledge of cryptography.

Most of the complex cryptographic processes such as key derivation, chaotic S-Box generation, authentication tag verification and encryption operations are performed automatically by the system. This reduces user complexity while ensuring strong security protection for sensitive data.

Additionally, the system increases user confidence in data security by ensuring that encrypted files remain confidential and protected from unauthorized access. The integrity verification mechanism further ensures that the data has not been modified or tampered with during transmission or storage. This makes the system suitable for individuals and organizations that require reliable and secure data protection solutions.

Economic Feasibility:

The proposed system is economically feasible because it primarily relies on **open-source tools and technologies**, which significantly reduces development and deployment costs. The use of Python programming language and open-source cryptographic libraries eliminates the need for expensive proprietary software or licenses.

The system also does not require specialized or high-end hardware to operate effectively. A standard computer system with moderate processing power and memory is sufficient to run the encryption and decryption processes efficiently. The chunk-based processing mechanism further ensures that large files can be handled without requiring excessive system resources.

Furthermore, the system can be easily integrated into existing platforms such as secure file storage systems, cloud-based services, or data protection frameworks without requiring major modifications to the existing infrastructure. This flexibility reduces implementation costs and ensures long-term sustainability of the encryption system while maintaining high levels of data security.

3. SYSTEM REQUIREMENTS SPECIFICATION

The System Requirements Specification (SRS) defines the technical and functional requirements necessary for the successful development, deployment, and operation of the DeepShield AES, A Dynamic S-Box based Enhancement for Secure File Encryption System. This section outlines the software environment, hardware infrastructure, and core functional capabilities required to ensure secure file encryption, reliable authentication and efficient backend operations.

3.1 Software Requirements:

DeepShield is implemented as a web-based backend encryption system, ensuring secure file handling and authenticated access.

The software requirements describe the development tools, programming technologies, frameworks, and supporting libraries required to build and execute the DeepShield system efficiently.

Operating System

The system is compatible with:

- Windows 10 or higher
- Linux distributions
- macOS

This ensures portability and cross-platform compatibility.

Programming Languages

The system uses the following programming languages:

- Python - Used for implementing the Dynamic S-Box based AES enhancement algorithm and cryptographic computations
- JavaScript (Node.js environment) - For backend logic handle file operations, and perform encryption processes efficiently.
- JavaScript (React.js) - For building the interactive user interface

- HTML5 - For structuring web pages
- CSS3 - For styling and responsive layout design

These technologies allow the development of a dynamic, responsive, and secure web application.

Frontend Framework

React.js is used to build the client-side interface of the DeepShield platform.

It is responsible for:

- Creating reusable UI components
- Managing application state
- Handling user interactions
- Displaying encryption and decryption activities

Backend Framework

Express.js is used to implement the backend server and REST APIs.

It is responsible for:

- Creating RESTful APIs
- Handling HTTP requests and responses
- Managing file encryption and decryption operations
- Implementing authentication middleware
- Processing file upload and download requests

Database Management System

MongoDB Atlas is used to store application data including:

- User credentials
- Encrypted file metadata
- File activity logs
- Encryption and decryption history

Mongoose provides:

- Schema-based data modeling
- Data validation
- Easy interaction between Node.js and MongoDB Atlas

Cloud Storage

Supabase is used for secure cloud file storage.

It is responsible for:

- Storing encrypted files
- Generating secure signed URLs for downloads
- Providing scalable and reliable file storage

File Handling

Multer is used for:

- Secure file upload functionality
- Handling multipart/form-data requests
- Temporary file storage during encryption

Environment Configuration

.env package is used to manage environment variables.

It helps to:

- Store sensitive configuration values
- Protect secret keys and API credentials
- Manage database connection strings
- Configure Supabase and JWT secrets securely

Development Tools

The following tools are used during development and testing:

- Visual Studio Code - Primary code editor used for development
- Postman - Used for API testing and debugging backend endpoints

- Git & GitHub - Used for version control and project collaboration

3.2 Hardware Requirements

The hardware requirements define the minimum and recommended physical infrastructure needed for smooth execution of encryption operations and backend services. DeepShield performs cryptographic operations such as encryption and decryption, which require moderate computational resources.

Processor

- Intel Core i5 (8th Generation or higher)
- AMD Ryzen 5 or higher

A multi-core processor ensures efficient encryption and server response handling.

Memory (RAM)

Minimum 8 GB RAM is required for:

- Running Node.js server
- Handling multiple API requests
- Processing large file encryption

Storage

Minimum 256 GB SSD (Recommended) storage ensures:

- Faster file read/write operations
- Reduced encryption processing time
- Efficient file upload/download handling

Network Requirements

Stable Internet connection is required for:

- Installing dependencies
- Connecting to MongoDB Atlas
- Deployment and updates

4. SYSTEM DESIGN

4.1 Introduction to System Design

System Design is an important phase in the development of any software application. It defines the overall architecture, system components, and the interactions between different modules of the proposed system. The purpose of system design is to convert the system requirements into a well-organized and structured solution that can be implemented efficiently.

In this project, the DeepShield AES system is designed to provide a secure and reliable platform for file storage and sharing. The design focuses on maintaining important system qualities such as security, scalability, efficiency and reliability. It supports various user operations including user registration, authentication, file uploading, encryption, secure sharing and downloading of files.

The platform ensures that all files uploaded by users are encrypted before being stored in cloud storage, which protects the confidentiality of the data. Only authorized users who possess the correct secure URL and password are allowed to access and decrypt the files. Any unauthorized access attempts are restricted to ensure the safety and integrity of the stored data.

To clearly illustrate the structure and working of the system, several design diagrams are used in this phase. These diagrams provide a visual representation of system interactions and data flow between different components. The main diagrams used in the system design include:

- Use Case Diagrams – to represent user interactions with the system
- Sequence Diagrams – to describe the step-by-step communication between system components
- Entity Relationship (ER) Diagrams – to illustrate the structure of the database and relationships between data entities

These diagrams help in understanding how different modules of the system interact

and how data flows through the application during various operations

4.2 System Architecture

4.2.1 Algorithm Architecture

The architecture of the proposed system describes how different components interact to perform secure file encryption, storage, and retrieval. The system follows a structured workflow that ensures data confidentiality, integrity, and controlled access. The architecture mainly consists of two major phases: Secure File Upload and Encryption and Secure File Retrieval and Decryption.

The system integrates cryptographic techniques, backend services, and cloud storage to provide a secure file management platform.

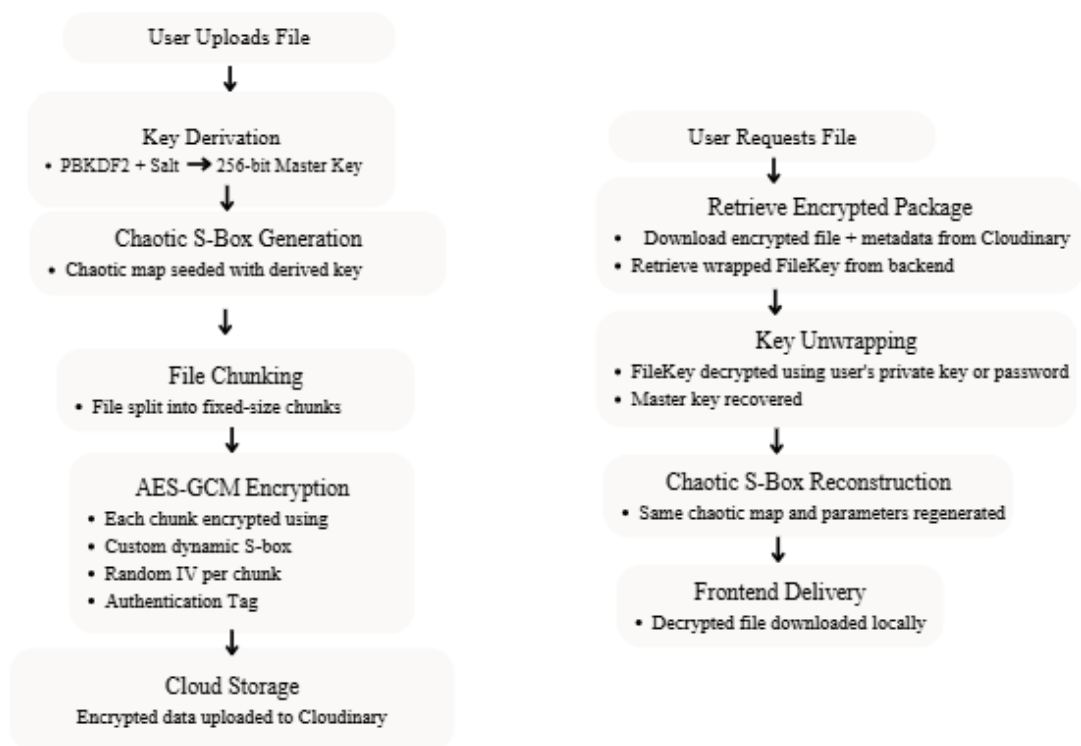


Fig: 4.2.1: Algorithm Architecture

4.2.1.1 File Upload and Encryption Process

The first stage of the system architecture focuses on securely encrypting user files before storing them in cloud storage.

1. User Uploads File

The process begins when a user uploads a file through the application interface. The user may also provide a password that will be used to generate a secure encryption key. This ensures that only users who know the password can decrypt the file later.

2. Key Derivation

After the file is uploaded, the system performs key derivation to generate a secure encryption key.

The system uses the PBKDF2 (Password-Based Key Derivation Function 2) algorithm along with a random salt value to derive a 256-bit master key. This process strengthens the encryption by preventing brute-force and dictionary attacks.

Key derivation ensures that:

- The encryption key is securely generated from the user password.
- The key is resistant to password guessing attacks.

3. Chaotic S-Box Generation

To improve the security of the encryption process, the system generates a Chaotic Substitution Box (S-Box).

The chaotic S-Box is generated using a chaotic map seeded with the derived master key. This creates a dynamic and unpredictable substitution structure that enhances the confusion property of the encryption algorithm.

Benefits of using a chaotic S-Box include:

- Increased randomness
- Stronger resistance against cryptographic attacks
- Enhanced security compared to static S-Boxes

4. File Chunking

Before encryption, the system divides the uploaded file into fixed-size chunks. Chunking provides several advantages:

- Efficient processing of large files
- Reduced memory usage
- Improved encryption performance

Each chunk is processed independently during encryption.

5. AES-GCM Encryption

Each file chunk is encrypted using AES (Advanced Encryption Standard) in GCM mode. The encryption process includes:

- Encryption of each chunk using the custom chaotic S-Box
- Generation of a random Initialization Vector (IV) for each chunk
- Creation of an Authentication Tag to ensure data integrity

AES-GCM provides both confidentiality and integrity protection, ensuring that the encrypted data cannot be modified without detection.

6. Cloud Storage

Once the encryption process is completed, the encrypted data is uploaded to cloud storage (Supabase).

Only encrypted files are stored in the cloud, ensuring that:

- Sensitive data remains protected
- Even if the cloud storage is compromised, the data cannot be accessed without the encryption key.

The system also stores related metadata and file references in the backend database.

4.2.1.2 File Retrieval and Decryption Process

The second stage of the architecture focuses on secure file access and decryption by authorized users.

1. User Requests File

When a user wants to access a stored file, they send a request through the application interface.

The system identifies the requested file and begins the retrieval process.

2. Retrieve Encrypted Package

The system retrieves the encrypted file package which includes:

- The encrypted file stored in cloud storage
- Metadata associated with the file
- The wrapped file encryption key stored in the backend system

The encrypted file is securely downloaded using a Supabase signed URL from the backend, while the password used to unwrap the key is provided by the frontend state and revealed only after successful encryption.

3. Key Unwrapping

To decrypt the file, the system performs key unwrapping.

The wrapped file encryption key is decrypted using the user's private key or password. Once this process is completed, the original master key used for encryption is recovered. This step ensures that only authorized users can access the decryption key.

4. Chaotic S-Box Reconstruction

After the master key is recovered, the system reconstructs the Chaotic S-Box using the same chaotic parameters and algorithms that were used during encryption.

This ensures that the exact substitution structure required for decryption is regenerated correctly.

5. Frontend Delivery

Once the file is successfully decrypted, the system delivers the original file to the user through the application interface.

The decrypted file is downloaded locally on the user's device, completing the secure file retrieval process.

The proposed architecture ensures secure file handling by combining:

- Advanced cryptographic techniques
- Dynamic chaotic S-Box generation
- AES-GCM encryption
- Chunk-based file processing
- Secure cloud storage

This layered security approach ensures that files remain encrypted during storage and transmission, protecting sensitive data from unauthorized access.

4.2.2 Application Architecture

The system architecture is designed to provide secure file encryption, storage and sharing while maintaining end-to-end confidentiality. The architecture primarily consists of three main components: the User (Client), Client-Side Encryption and Decryption Module and Cloud Storage System.

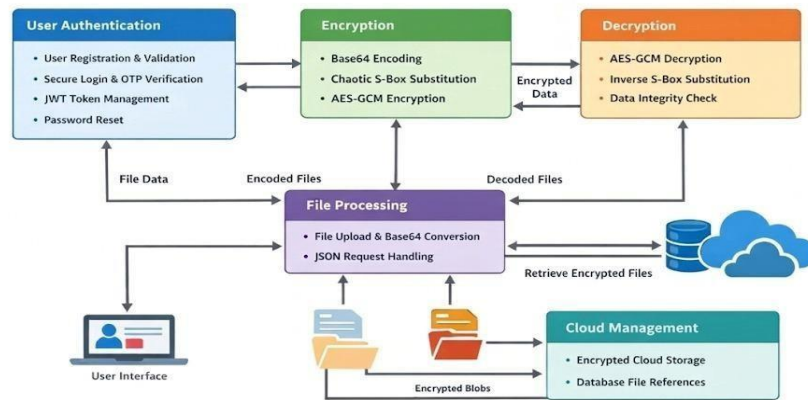


Fig: 4.2.2 Application Architecture

1. User Interaction

The process begins when a user accesses the DeepShield web application through a browser. The user uploads a text file and provides a password that will be used to encrypt the file. The password acts as the key for generating a secure encryption key.

2. Client-Side Encryption

Once the file and password are provided, the encryption process takes place entirely within the user's browser. This ensures that the original file is never exposed to the server in plain text form. The encryption module performs several operations:

PBKDF2 Key Derivation: The user's password is converted into a strong cryptographic key using the PBKDF2 algorithm.

Dynamic S-Box Generation: A unique substitution box (S-Box) is generated for every encryption session, improving security by making the encryption structure dynamic.

AES-GCM Encryption: The file is encrypted using the AES-GCM encryption mode, which ensures both confidentiality and integrity of the data.

After encryption is completed, the file becomes an encrypted file that cannot be read without the correct password.

3. Cloud Storage

The encrypted file is then uploaded to a secure cloud storage platform. The system generates a unique URL that points to the encrypted file stored in the cloud. Since the file is already encrypted before uploading, the cloud server only stores encrypted data and cannot access the original content.

4. Secure Sharing

The user can share the generated secure URL along with the password with authorized recipients. Both pieces of information are required to access and decrypt the file, ensuring controlled access.

5. Authorized User Decryption

When an authorized user receives the URL and password, they can access the encrypted file through the application. The encrypted file is downloaded and decrypted directly in the browser using the same DeepShield AES process. If the correct password is entered, the file is successfully decrypted and made available for viewing or download.

6. Security Advantage

This architecture ensures that:

- Encryption and decryption occur only on the client side.
- The server and cloud storage never see the original file.
- Only users with both the secure URL and correct password can access the data.

Thus, the DeepShield system architecture provides a secure, scalable and privacy-preserving file sharing system.

4.3 UML Diagrams

Unified Modeling Language (UML) diagrams are standardized graphical representations used in software engineering to model, visualize, and document the structure and behavior of a system. UML provides a common language that helps developers, designers, and stakeholders understand how a system operates and how its components interact with each other. These diagrams act as a blueprint for system development, making it easier to design complex software systems in a structured and organized manner.

UML diagrams help in representing different aspects of a system such as system architecture, data flow, object relationships, and interactions between components. By providing a visual representation, UML diagrams simplify the process of understanding system functionality and communication between modules. They are widely used in system analysis, software design, and documentation because they provide a clear and standardized method for describing software systems.

Key Categories

UML diagrams are generally categorized into two main groups: Structural Diagrams and Behavioral Diagrams. Structural diagrams focus on the static components of the system and describe how different elements are organized within the system architecture. These diagrams represent the structure of the system, including classes, objects, components, and their relationships. Some commonly used structural diagrams include Class Diagrams, Component Diagrams, Deployment Diagrams, Object Diagrams, and Package Diagrams.

Behavioral diagrams, on the other hand, describe the dynamic aspects of the system and illustrate how different components interact with each other during system execution. These diagrams help in understanding the workflow of the system and how data moves between different processes. Examples of behavioral diagrams include Use Case Diagrams, Sequence Diagrams, Activity Diagrams, and State Machine Diagrams. These diagrams show how users interact with the system and how various operations are performed step by step.

In software development, UML diagrams are commonly used for system design, business process modeling, and system documentation. They help developers plan the

architecture of a system, identify relationships between system components, and visualize workflows before actual implementation begins. This improves system clarity and reduces design errors during development.

UML diagrams can also be used for database modeling in modern software systems. In such cases, UML Class Diagrams are often used to represent database tables and their relationships. Classes correspond to database tables, attributes represent table columns, and associations illustrate relationships between tables. This approach is particularly useful in object-oriented systems where the application structure closely relates to the database structure.

Although Entity-Relationship (ER) diagrams are traditionally used for database design, UML diagrams provide a broader view by representing both the system structure and the interaction between application components and the database. Because of this advantage, UML diagrams are widely adopted in modern system design methodologies.

In this project, UML diagrams are used to illustrate the interactions between different components of the DeepShield AES system, including users, the web application, backend services, database, and cloud storage. These diagrams provide a clear understanding of how files are uploaded, encrypted, stored, retrieved, and decrypted within the system.

Use Case Diagram

The Use Case Diagram represents the interactions between users and the system. It shows the different functionalities available in the system and how different types of users interact with them.

The system consists of three primary actors:

- 1. User:** A normal user who interacts with the system to register, authenticate, and upload files.
- 2. Authorized User:** An authenticated user who has permission to upload, share, decrypt, and access encrypted files.
- 3. Unauthorized User:** A user who tries to access the system without proper authorization.

Main Use Cases

1. **Register Account:** Allows a new user to create an account in the system.
2. **Authenticate User:** Verifies the identity of a user before allowing access to system services.
3. **Upload Text File:** Users can upload files to the platform for secure storage.
4. **Encrypt File:** Before storing the file, it is encrypted using the AES encryption mechanism to ensure data confidentiality.
5. **Generate Unique Access URL:** After encryption, a unique secure URL is generated which can be shared with other users for accessing the file.
6. **Host Encrypted File on Cloud:** Encrypted files are stored securely in cloud storage.
7. **Decrypt File:** Authorized users can decrypt the file using the correct password.
8. **Share File Access Information:** Users can share the generated access URL with others to allow secure access.
9. **Attempt Unauthorized Access:** Unauthorized users attempting to access the system are restricted.

The Use Case Diagram helps to visualize the overall system functionality and user interactions.

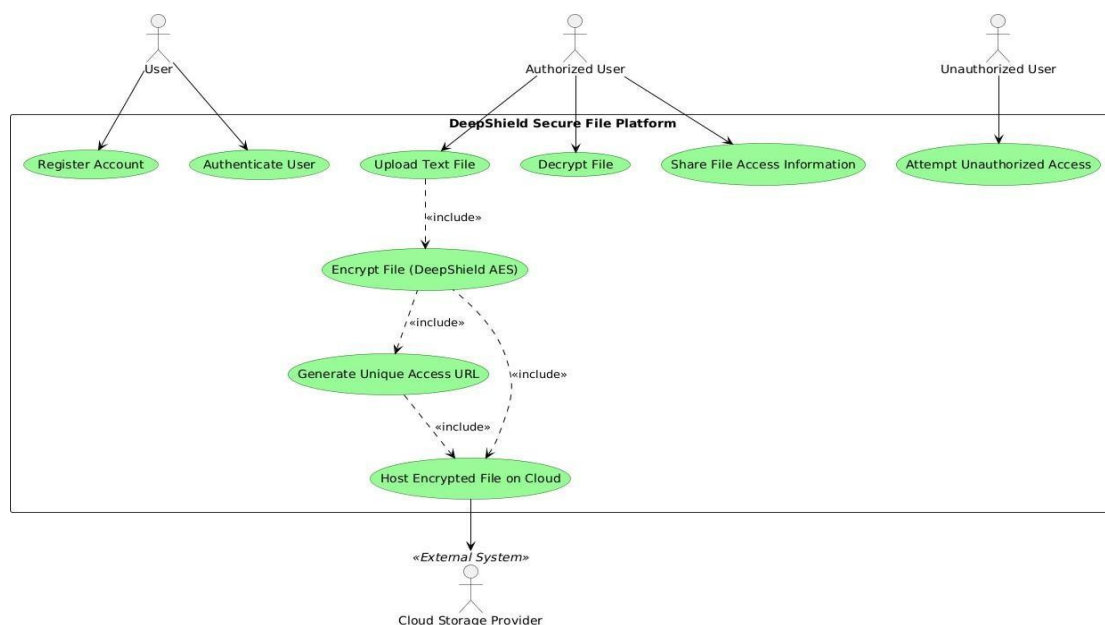


Fig: 4.3.1 Use Case Diagram

Sequence Diagram

The Sequence Diagram describes the step-by-step interaction between different system components during file upload and download operations.

The major components involved are:

- User
- Web Browser (Client)
- Backend Server
- Database
- Cloud Storage

Secure File Upload and Encryption

1. The user authenticates into the system.
2. The user selects a file and enters an encryption password.
3. The web browser reads the file content into memory.
4. The system generates a unique Initialization Vector (IV) and salt.
5. The file is encrypted using the DeepShield AES encryption algorithm.
6. The encrypted file upload request is sent to the backend server.
7. The server stores file metadata in the database.
8. The backend requests a pre-signed upload URL from cloud storage.
9. The cloud service returns the upload URL.
10. The encrypted file is uploaded directly to cloud storage.
11. The system confirms successful upload.
12. The server records the upload completion with a file ID.
13. The system generates and displays a secure file URL to the user.
14. The user can share the URL with others along with the password.

Secure File Decryption and Download

1. The recipient receives the file access URL and password.

2. The user opens the access URL.
3. The system extracts the file ID from the URL.
4. The backend validates user authorization and retrieves metadata.
5. The backend requests a pre-signed download URL from cloud storage.
6. The cloud storage returns the download link.
7. The encrypted file is downloaded.
8. The user enters the encryption password.
9. The system decrypts the file.
10. The decrypted file is made available for download.

This sequence ensures that files remain encrypted during storage and transfer, maintaining strong security.

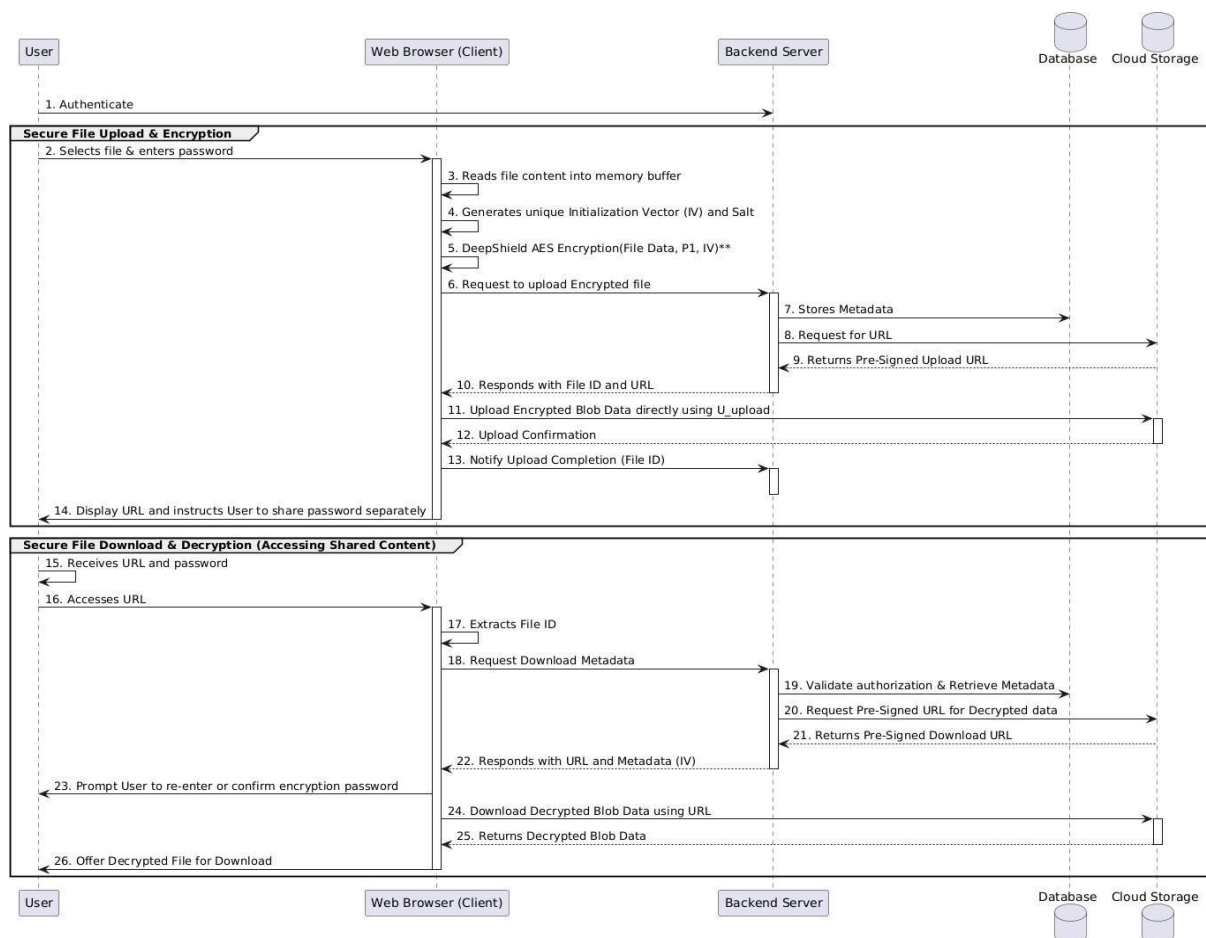


Fig: 4.3.2 Sequence Diagram

4.4 Database Design

The database design defines how data related to users, files, encryption keys, and system activities are stored and managed. The database is structured using entities and relationships to ensure efficient data storage and retrieval.

The system database consists of four main tables:

- Users
- Files
- Decryption Logs

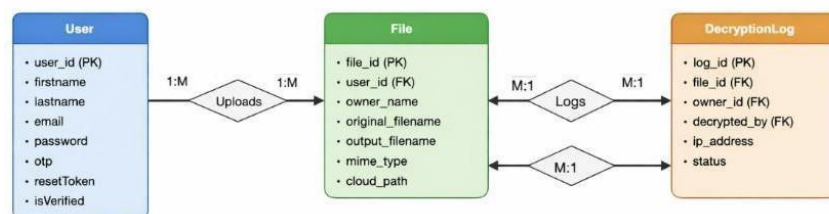


Fig: 4.4.1 Database Design

The User entity stores information about registered users of the system, including their personal details, login credentials, verification status, OTP information, and password reset tokens. Each user is uniquely identified by a user ID.

The File entity stores metadata related to files uploaded by users. Each file record includes details such as the original file name, encrypted output file name, MIME type and the cloud storage path where the encrypted file is stored. Every file is associated with a specific user through the user_id foreign key, indicating that a single user can upload multiple files.

The DecryptionLog entity maintains a record of all file decryption activities for auditing and security purposes. It records which file was decrypted, the owner of the file, the user who performed the decryption, the IP address of the request, and the status of the operation. This helps in monitoring unauthorized access attempts and maintaining accountability within the system.

The relationships in the diagram indicate that one user can upload many files, and one file can have multiple decryption logs, representing different decryption attempts. This structure ensures proper tracking of user activities and secure management of encrypted files within the DeepShield system.

5. SYSTEM IMPLEMENTATION

5.1 Introduction

The DeepShield system is implemented as a secure web-based application designed to protect user files through encryption-driven architecture. The implementation focuses on ensuring confidentiality, controlled access, and secure cloud storage integration.

The application follows a client-server model. The frontend provides an interactive and responsive user interface for registration, authentication, file upload and file retrieval. The backend processes user requests, performs encryption and decryption operations, manages authentication tokens and interacts with the database and cloud storage. Before any file is stored in the cloud, it is encrypted using DeepShield AES. This ensures that even if unauthorized access occurs at the storage level, the file content remains unreadable without proper authorization.

The implementation emphasizes:

- Secure authentication mechanisms
- Controlled file access
- Encryption before storage
- Activity tracking for monitoring
- Scalable architecture for future enhancements

The system is modular in design, making it maintainable and extensible.

5.2 Project Modules

This web application is organized into logically separated modules to ensure maintainability, scalability, and security. Each module is responsible for a specific set of operations within the application. The modular design allows independent development and testing of system components while maintaining overall integration

- 1. User Authentication Module** is responsible for verifying the identity of users and controlling access to system resources. It ensures that only authorized

users can access encryption and decryption services. By implementing multi-layer verification and token-based authentication, the system prevents unauthorized access and session misuse.

Functionality:

- User registration with input validation
- Secure password storage using hashing techniques
- Login authentication using credentials
- One-Time Password (OTP) verification through registered email
- Token-Based session management (JWT)
- Password reset using secure email-based token

2. Encryption Module is the core security engine of DeepShield. It transforms readable file data into a protected encrypted format before storage or transmission. This module ensures both confidentiality and integrity of stored files.

Functionality:

- Accepts base64-encoded file data from the frontend
- Text is converted into base64-encoded data
- Converts base64 data into raw binary format
- Applies byte-level substitution using chaotic S-Box
- Encrypts substituted data using AES in GCM mode
- Returns encrypted data as base64 for safe JSON transport

3. Decryption Module restores encrypted content to its original text using secure reverse operations.

Functionalities:

- Accepts encrypted base64 data
- Regenerates identical chaotic S-Box
- Constructs inverse S-Box for reverse substitution
- Performs AES-GCM decryption
- Verifies authentication tag for integrity validation
- Applies inverse substitution to recover original file bytes

- Returns decrypted content in base64 format to frontend

4. File Processing Module manages data transformation between frontend and backend over HTTP without binary corruption.

Functionalities:

- Accepts text data in the form of files from the user
- Converts uploaded files into base64 format (frontend)
- Decodes base64 into raw bytes (backend)
- Encodes encrypted binary output into base64 for JSON response
- Handles structured JSON request and response objects

5. Cloud Management Module handles interaction with external cloud services and storage operations. By storing only encrypted files in the cloud, this module minimizes security risks. The separation of encryption logic and storage management strengthens overall system protection.

Functionalities:

- Stores encrypted file blobs in external storage
- Maintains separation between encryption logic and storage layer
- Stores cloud file references in the database
- Retrieves encrypted blobs for decryption
- Prevents storage of plain-text file content

5.3 Algorithm Explanation

Encryption Process

DeepShield AES introduces additional dynamic complexity using an enhanced AES- based encryption framework combining:

- PBKDF2 Key Derivation
- Chaotic S-Box Generation
- AES-256 in GCM Mode

Step 1: Key Derivation Using PBKDF2

Instead of directly using the password as the AES key, the system applies:

- PBKDF2 with SHA-512
- 100,000 iterations
- 16-byte random salt

This produces a secure 256-bit key.

Purpose:

- Protects against brute-force attacks
- Prevents rainbow table attacks
- Ensures unique key generation for each encryption

Step 2: Chaotic S-Box Generation

A dynamic substitution box is generated using a logistic chaotic map:

$$X_{n+1} = r \cdot X_n \cdot (1 - X_n)$$

Where:

- $r = 3.99$ (chaotic parameter)
- Seed derived from first 4 bytes of AES key

Process:

1. Generate 256 chaotic values
2. Sort values
3. Create byte substitution mapping

Purpose:

- Produces a key-dependent substitution layer
- Adds unpredictability
- Prevents static substitution patterns
- Increases resistance to cryptanalysis

Step 3: Byte Substitution

Each byte of file data is replaced using the generated chaotic S-Box. This introduces non-linearity before AES encryption.

Step 4: AES-256 Encryption in GCM Mode

After substitution, AES encryption is applied using:

- 256-bit key
- GCM (Galois/Counter Mode)
- 12-byte nonce

Purpose

AES-GCM provides:

- Confidentiality
- Authentication
- Integrity verification
- Tamper detection

Step 5: Secure Blob Formation

Final encrypted file structure:

Salt (16B) | Nonce (12B) | Tag (16B) | Cipher Text

This combined structure ensures that the decryption process can accurately reconstruct the original key and decrypt the file securely.

Decryption Process

DeepShield AES reverses the encryption pipeline by reconstructing the key, regenerating the chaotic S-Box, and applying authenticated AES-256 decryption to restore the original file.

The decryption framework consists of the following stages:

- PBKDF2 Key Reconstruction
- Chaotic S-Box Regeneration

- Reverse Byte Substitution
- AES-256 GCM Decryption

Step 1: Extract Encrypted Blob Components

The encrypted file contains the following structure:

Salt (16B) | Nonce (12B) | Tag (16B) | Cipher

Text The system first separates these components.

Purpose:

- Retrieve parameters required for key reconstruction
- Obtain nonce and authentication tag for AES-GCM verification
- Extract ciphertext for decryption

Step 2: Key Reconstruction Using PBKDF2

The user-provided password is processed through the same key derivation algorithm used during encryption:

- PBKDF2 with SHA-512
- 100,000 iterations
- 16-byte extracted salt

This regenerates the original 256-bit AES key.

Purpose:

- Reproduce the exact encryption key
- Ensure secure password-based authentication
- Maintain resistance to brute-force attacks

Step 3: Chaotic S-Box Regeneration

Using the regenerated AES key, the system reconstructs the same dynamic S-Box.

Process:

1. Generate 256 chaotic values
2. Sort the generated values

3. Construct the substitution mapping
4. Compute the inverse S-Box

Purpose:

- Recreate the same substitution layer used in encryption
- Enable correct reversal of substituted byte values

Step 4: AES-256 Decryption in GCM Mode

The ciphertext is decrypted using:

- 256-bit AES key
- Extracted 12-byte nonce
- Authentication tag

Purpose:

- Restore substituted data
- Detect tampering or corruption
- Ensure integrity and authenticity of the encrypted file

If the authentication tag fails verification, the decryption process is aborted.

Step 5: Reverse Byte Substitution

After AES decryption, the substituted byte data is obtained. The system applies the inverse chaotic S-Box to each byte.

Purpose:

- Reverse the substitution transformation
- Recover the original file byte sequence

Step 6: File Reconstruction

The recovered byte stream is converted back to its original format and returned to the frontend.

Output:

- Decrypted file in Base64 format
- Frontend decodes Base64 to reconstruct the original file.

5.4 Screens

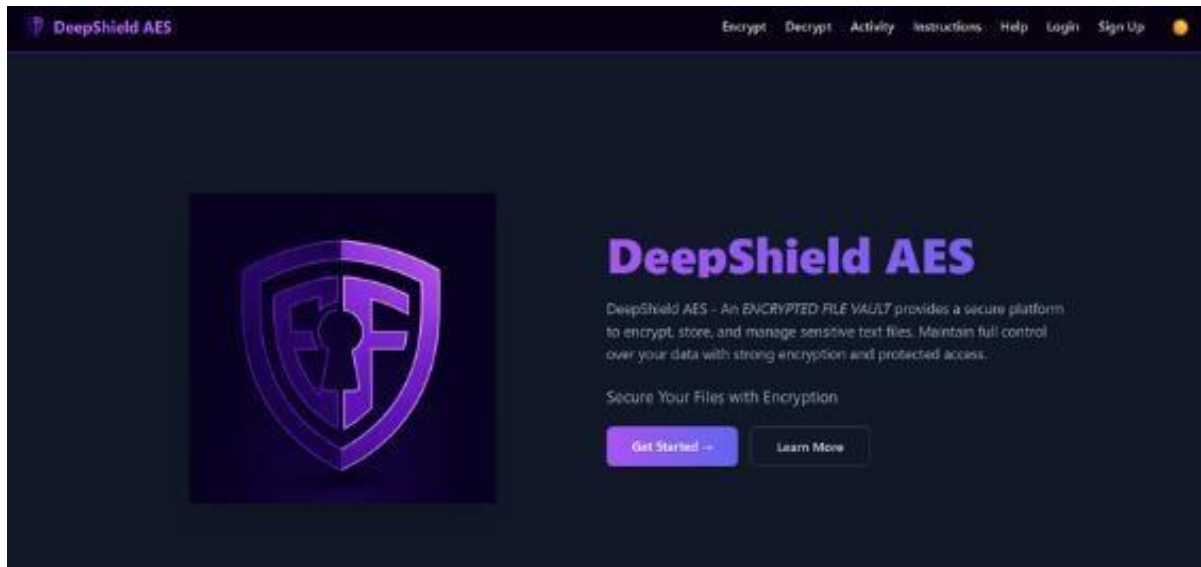


Fig: 5.4.1 Home Page

The **Home Page** serves as the main entry point of the DeepShield application. It introduces users to the platform and provides an overview of the secure file encryption system. The page is designed with a clean and responsive interface to create a professional first impression and guide users toward accessing the application features. This page contains a hero section that displays the web application title "DeepShield" along with a brief description explaining the purpose of the system. It highlights that the application provides a secure digital environment for uploading, encrypting and storing sensitive files. This section also includes a "Get Started" button that directs users to the login page.

The page uses modern UI technologies such as React, Tailwind CSS, and Framer Motion to provide smooth animations and responsive design. The layout is structured to maintain readability and accessibility across different devices, including desktops and mobile screens.

Additionally, the home page integrates sections such as Security Features, Usage Instructions, Help Section and Footer. These sections provide important information about the system's functionality and assist users in understanding how to securely use the application. Overall, the Home Page acts as an introduction to the Deep Shield platform and helps users easily navigate to the authentication and file encryption interfaces.

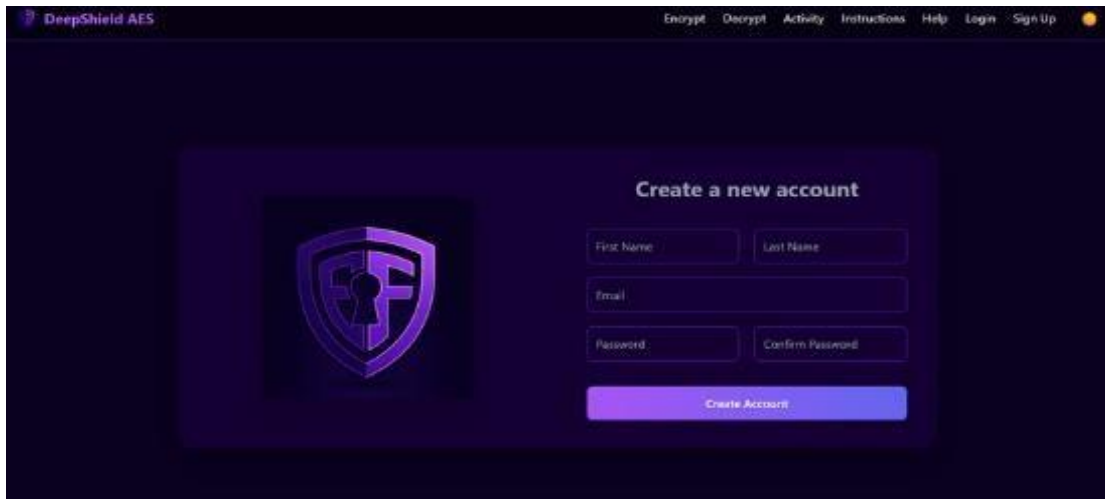


Fig: 5.4.2 SignUp Page

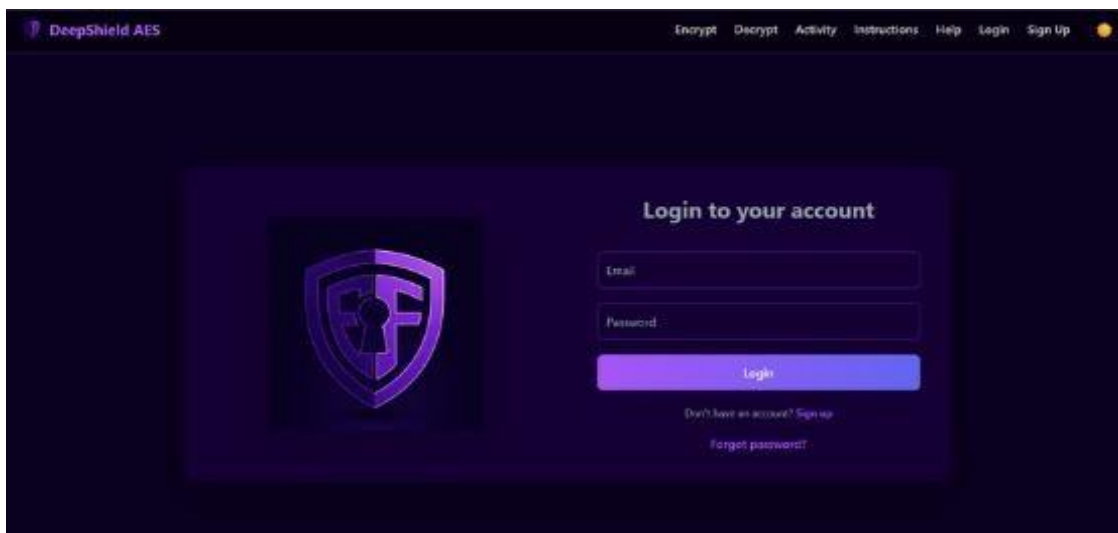


Fig: 5.4.3 Login Page

The **Login and Sign Up pages** allow users to securely access the system.

- Sign Up: New users can register by providing required details such as name, email and password.

- Login: Existing users can log in using their registered email and password navigating to OTP verification page for multi-factor authentication.

These pages ensures that only authenticated users can access the encryption and decryption services provided by the system.

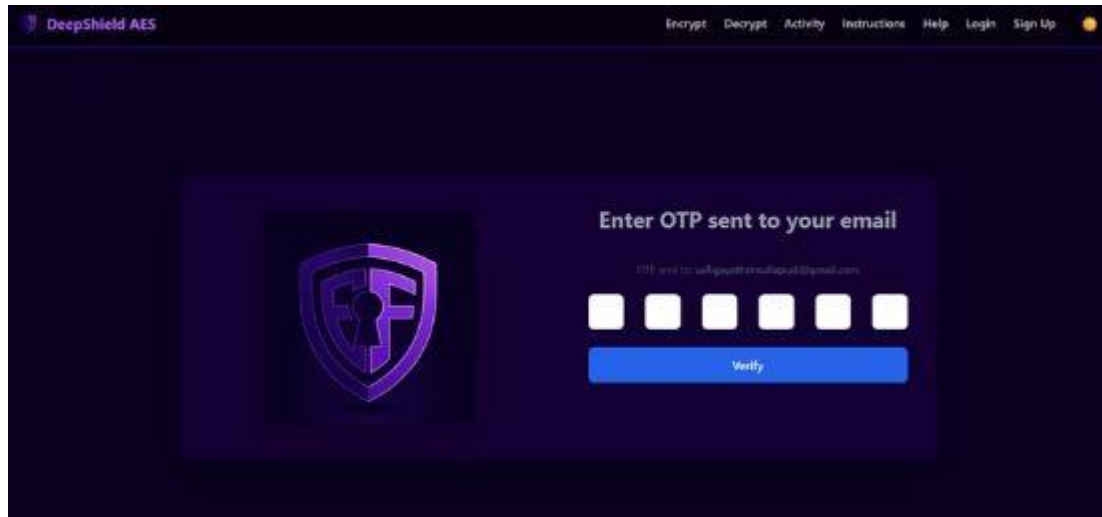


Fig:5.4.4 OTP Verification Page

The **OTP (One-Time Password) verification page** provides an additional layer of security during user authentication. After entering login credentials, a unique OTP is sent to the user's registered email. The user must enter the OTP correctly within 5 minutes from the time OTP generated to verify their identity and gain access to the system. This helps prevent unauthorized access.



Fig: 5.4.5 Dashboard

The **Dashboard** acts as the main control panel of the system after a user logs in. It provides quick access to the major functionalities of the application such as Encrypt, Decrypt, Activity Logs and Help/Instructions. The dashboard allows users to easily navigate between different features of the system.

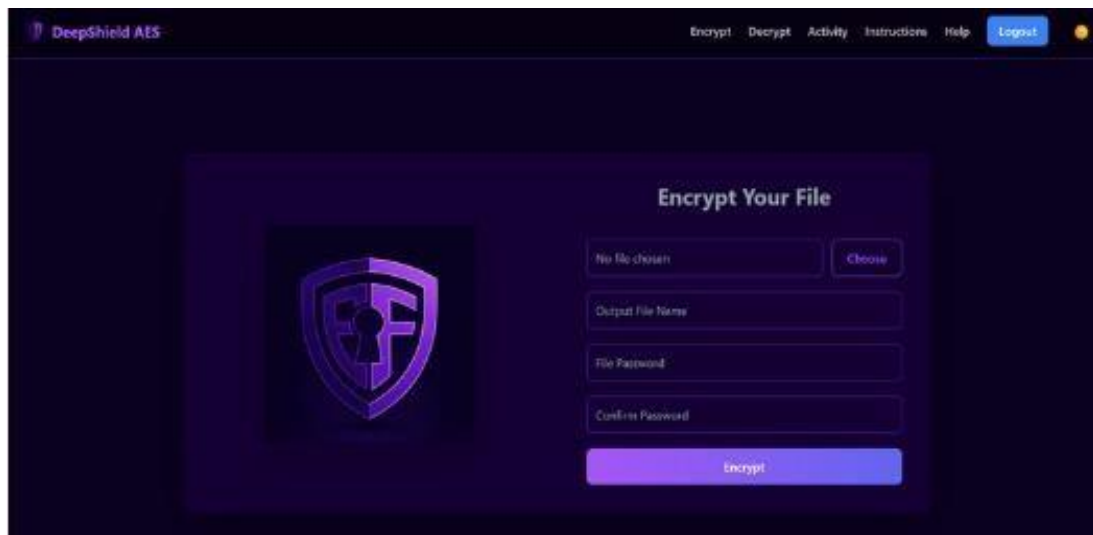


Fig: 5.4.6 Encrypt Page

The **Encrypt Page** allows users to securely encrypt their files. Users first choose a file from their device and optionally provide an output file name. To protect the file, the user must enter a password and confirm it. The system checks whether the password is strong enough. A "Suggest Strong Password" option is also provided to automatically generate a secure password.

After filling the details, the user clicks the Encrypt button. The system then encrypts the selected file and stores it securely in the Supabase cloud storage. Once encryption is successful, the system generates a URL for the encrypted file. This URL indicates the file path in the cloud storage. The user can copy the file URL and password using the copy buttons. A warning message is also displayed to remind the user to save the password safely, because it cannot be recovered later.

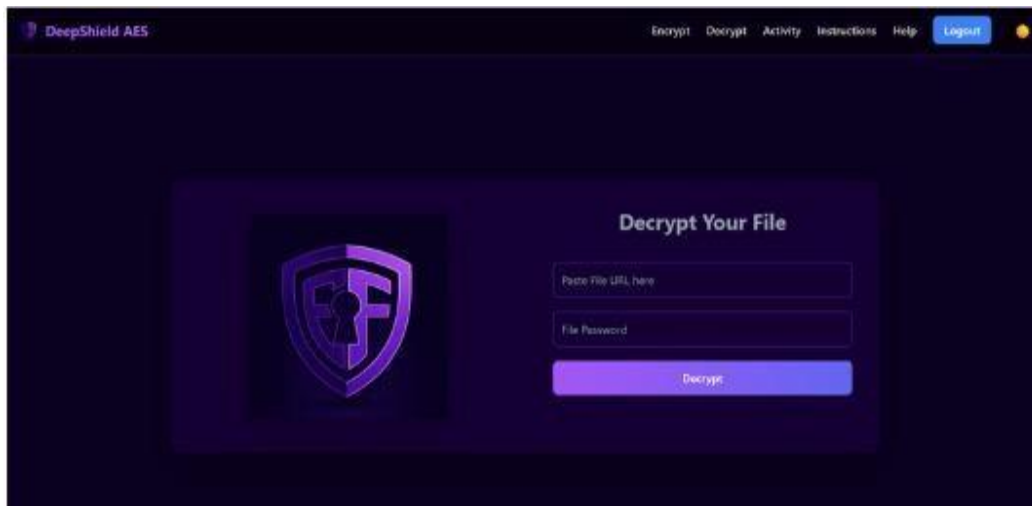


Fig: 5.4.7 Decrypt Page

The **Decrypt page** allows users to restore previously encrypted files back to their original format. Users must upload the encrypted file URL and provide the correct password used during encryption. If the password is correct, the system decrypts the file and allows the user to download the original file. Otherwise, the decryption process will fail and the system displays an error "Decryption Failed".

File Name	URL	Encrypted Time
intro.txt	@9ef14807a3d76d9732e602_1773268274790_intro.txt.enc	11/1/2025, 10:32:56 pm
intro.txt	@9ef14807a3d76d9732e602_1773267940632_intro.txt.enc	11/1/2025, 10:30:44 pm
intro.txt	@9ef14807a3d76d9732e602_1773136207957_intro.txt.enc	10/31/2025, 3:21:29 pm
intro.txt	@9ef14807a3d76d9732e602_1773132145831_intro.txt.enc	10/31/2025, 2:10:07 pm
intro.txt	@9ef14807a3d76d9732e602_1773131325177_intro.txt.enc	10/31/2025, 1:58:45 pm
intro.txt	@9ef14807a3d76d9732e602_1773131118932_intro.txt.enc	10/31/2025, 1:55:38 pm
intro.txt	@9ef14807a3d76d9732e602_1772966612747_intro.txt.enc	7/3/2025, 12:26:54 pm
intro.txt	@9ef14807a3d76d9732e602_1772966280077_intro.txt.enc	7/3/2025, 12:21:02 pm

Fig: 5.4.8 Activity Page

The **Activity Page** displays the history of all file operations performed in the system. It shows details about files that the user has encrypted and decrypted, along with the time of each action. The page also indicates if other users have decrypted the user's files. This helps users track their file usage and monitor access to their encrypted files. It improves transparency and security within the system.

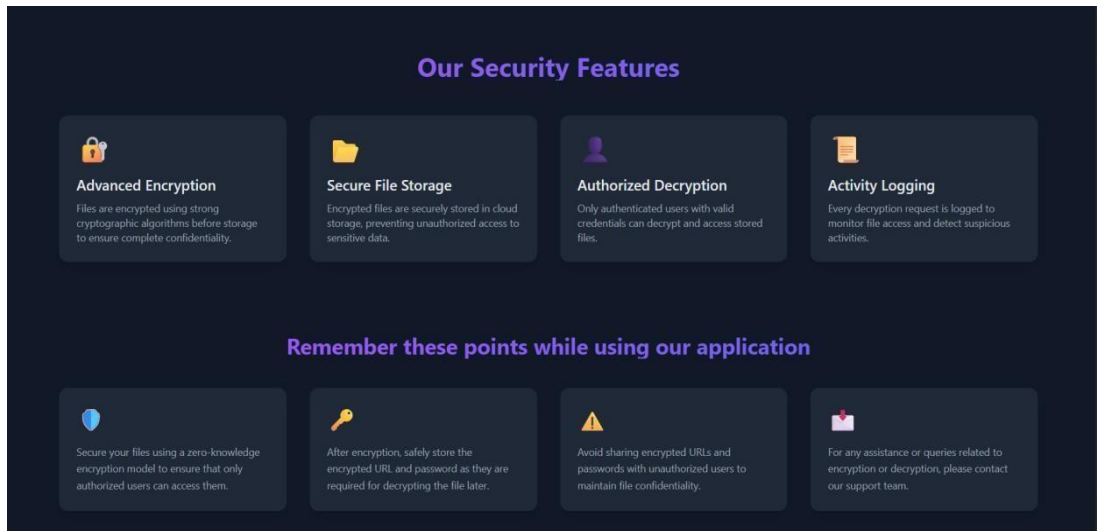


Fig: 5.4.9 Instructions Page

The **Instructions Page** provides detailed information about the security features of the Deep Shield system and offers guidelines for users to securely handle encrypted files. This page is designed to educate users about how the encryption system works and how to maintain the confidentiality of their data.

The page is divided into two main sections: Security Features and Security Guidelines.

The page also includes a Security Reminder that emphasizes the importance of keeping encryption passwords secure, as losing the password may permanently prevent access to encrypted files.

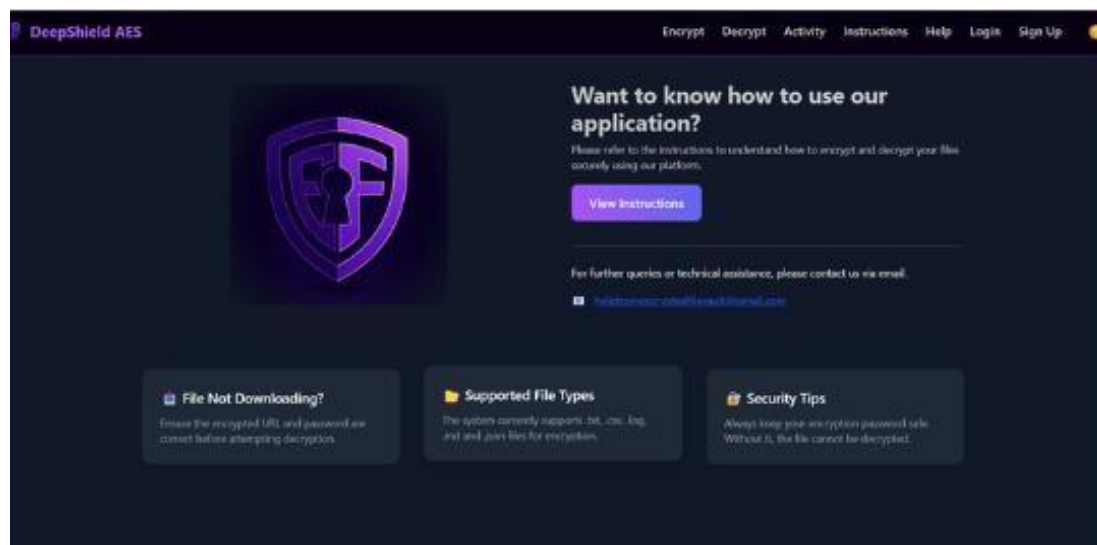


Fig: 5.4.10 Help Page

The **Help Section** provides guidance to users on how to use the application effectively. It includes a link to view detailed instructions for encrypting and decrypting files. Users can also contact support through the provided email for technical assistance. The section displays useful tips such as supported file types and solutions for common issues. It also reminds users to keep their encryption passwords safe for security.

5.5 System Deployment

The application follows a distributed deployment architecture where the frontend and backend components are hosted on separate platforms. This approach allows independent development, deployment, and maintenance of each component while improving overall system efficiency.

The frontend of the application is deployed using Vercel, a cloud platform widely used for hosting modern web applications. Vercel provides fast content delivery through a global Content Delivery Network (CDN), ensuring that users can access the application with minimal latency. The ReactJS-based user interface is hosted on Vercel, which automatically builds and deploys the application whenever updates are pushed to the project repository. This allows seamless integration between development and deployment, enabling continuous updates and improved user experience.

The backend server of the system is deployed using Render, a cloud platform designed for hosting backend services and APIs. The backend application, which includes the DeepShield AES encryption and decryption algorithm, runs on the Render server. This server handles core functionalities such as user authentication, file encryption, file metadata management, secure file access control, and communication with cloud storage services. Render provides reliable server infrastructure, automatic scaling capabilities, and secure HTTPS connections for API communication.

During the file upload process, the frontend application hosted on Vercel sends requests to the backend API hosted on Render. The backend server then performs encryption using the DeepShield AES algorithm before securely storing the encrypted files in cloud storage. Similarly, when a user requests to download a file, the backend retrieves the encrypted file, verifies authorization, and performs the decryption process before delivering the file to the user.

During the development phase, the DeepShield AES system was initially implemented and tested in a local development environment (localhost). The frontend

application was executed locally using the ReactJS development server, while the backend services, including the DeepShield AES encryption algorithm and API endpoints, were run on a local server environment. This setup allowed efficient debugging, testing of system functionalities, and validation of the encryption and decryption processes before deploying the application to a public hosting environment.

After successful testing in the localhost environment, the system was deployed to cloud platforms to make it accessible over the internet. The frontend of the application was hosted on Vercel, while the backend server containing the DeepShield AES encryption logic was deployed on Render. By deploying the frontend and backend on separate cloud platforms, the system benefits from improved scalability, enhanced performance, and better resource management.

This deployment architecture allows the frontend application hosted on Vercel to communicate securely with the backend services running on Render through API requests. Such separation of services improves maintainability and allows independent updates to each component without affecting the overall system functionality. Additionally, the use of cloud platforms ensures that the application can handle future updates and increased user traffic efficiently while maintaining strong security for encrypted file management.

6. SYSTEM TESTING

6.1 Introduction

Software testing is a structured process carried out to evaluate the correctness, reliability, performance, and security of an application. It ensures that the developed system satisfies specified requirements and operates without unexpected failures. Testing is not limited to identifying errors; it also validates that the software behaves as intended under both normal and abnormal conditions.

In security-focused applications such as DeepShield AES system, testing plays a critical role. Since the system handles sensitive file encryption and decryption processes, even a minor flaw could compromise data confidentiality or integrity. Therefore, systematic testing was conducted at multiple levels to verify the accuracy of cryptographic operations, API responses, data transmission and user interactions.

The testing process in our project focused on the following aspects:

- Correctness of AES encryption and decryption logic
- Proper functioning of password-based key derivation
- Verification of authentication and access control
- Data integrity during encryption and transmission
- Secure handling of base64 encoding and decoding
- Error handling for incorrect passwords or corrupted data

Testing was performed throughout the development lifecycle. Each module was validated independently before integration into the complete system. After integration, the entire workflow from file upload to encrypted storage and secure decryption was tested under various scenarios.

Particular attention was given to:

- Ensuring that encrypted outputs differ even for identical files (due to random salt and nonce)
- Verifying that decryption fails when incorrect passwords are provided

- Confirming that any modification in encrypted data results in authentication failure (AES-GCM integrity check)
 - Ensuring proper reconstruction of original files at the frontend after decryption
- By performing systematic and multi-level testing, the reliability and robustness of the system were validated before deployment.

6.2 Testing Methods

Different testing methodologies were applied to ensure comprehensive system validation.

- **White Box Testing** focuses on the internal logic and structure of the code. Since the source code is accessible, internal functions and control flow paths were examined. White box testing was applied to:
 - Key derivation function using PBKDF2
 - Chaotic S-Box generation logic
 - AES encryption and decryption functions
 - Extraction of salt, nonce, tag, and ciphertext
 - Error handling blocks

Each logical branch was verified to ensure correct execution paths. Special attention was given to verifying that decryption only succeeds when authentication tags match.

- **Black Box Testing** evaluates system functionality without considering internal implementation.

This included:

- Testing encryption endpoint using valid and invalid inputs
- Testing decryption endpoint with correct password
- Testing decryption with wrong password
- Testing with modified encrypted data
- Verifying API responses and error messages

The objective was to ensure that the system behaves correctly from a user perspective.

- **Unit Testing** involves testing individual functions independently. The following units were tested:
 - `derive_key()` function

- generate_chaotic_sbox() function
- encrypt_data() function
- decrypt_data() function

Each function was executed with sample inputs to confirm correct outputs.

- **Integration Testing** verifies interaction between modules. It mainly involves:
 - Frontend file upload → backend encryption
 - Backend encryption → base64 encoding → frontend reception
 - Frontend request → backend decryption → file reconstruction
 - JSON request and response formatting

The full workflow was tested to ensure seamless data flow between client and server.

- **Validation Testing** ensures that the system meets user requirements. Validation was performed to confirm:
 - Files are correctly restored after decryption
 - No data loss occurs during processing
 - Encryption output varies for identical files (due to random salt)
 - Authentication fails when incorrect password is entered
 - System provides appropriate error messages

This confirmed that DeepShield AES system satisfies its security and functional objectives.

6.3 Test Cases

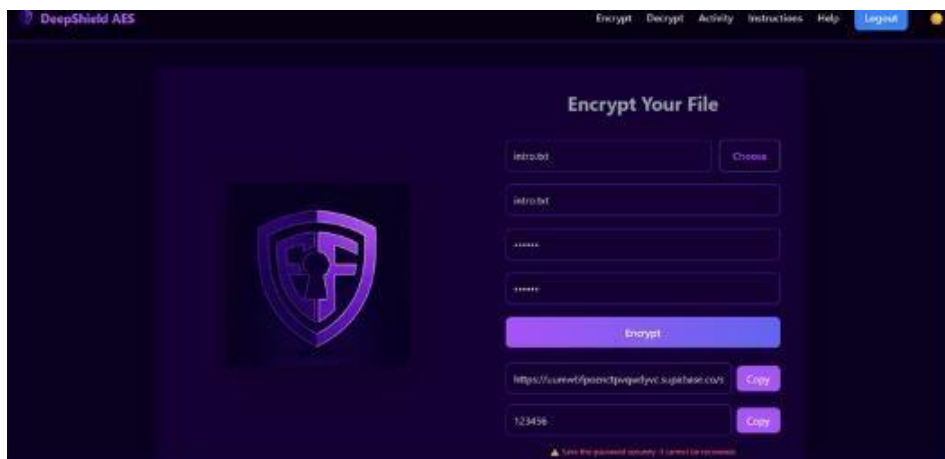


Fig: 6.3.1 Successful Encryption

Test Case 1: Successful Encryption

- **Test Input:** Upload a valid file along with a valid password.
- **Expected Outcome:** The system should successfully encrypt the file using the DeepShield AES algorithm and return URL of the encrypted content in base64 format.
- **Actual Result:** The encryption process completed successfully and URL for the encrypted file was generated.
- **Status:** Pass

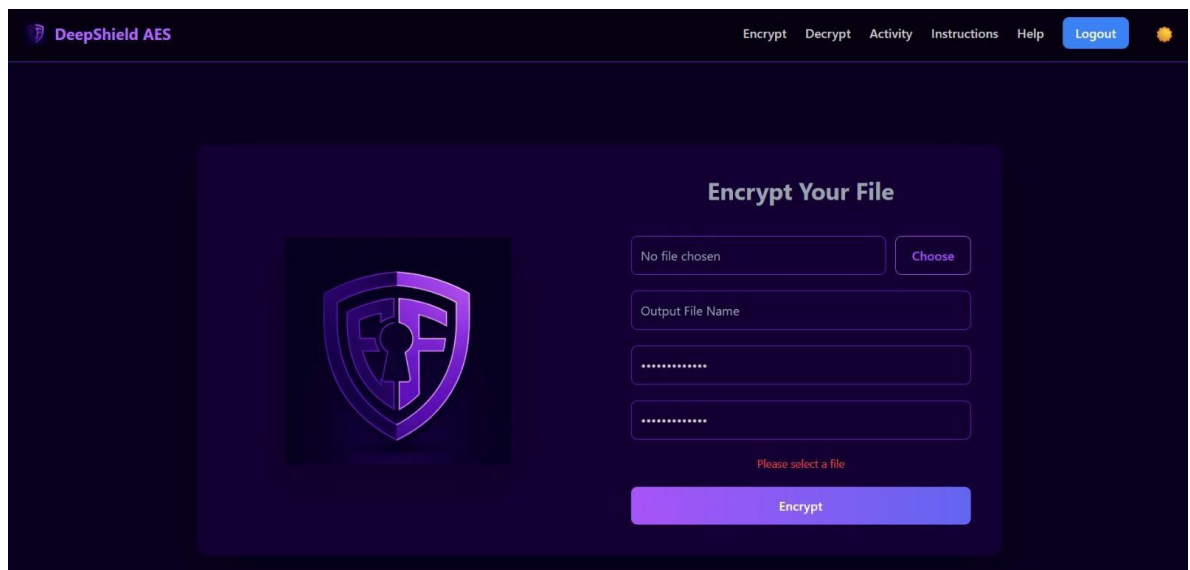


Fig: 6.3.2 Encryption with missing data

Test Case 2: Encryption with Missing Data

- **Test Input:** Send an encryption request without providing file data or password.
- **Expected Outcome:** The system should detect the missing input and return an error message indicating that required data or password is missing.
- **Actual Result:** The system returned an error response stating "Please select a file"
- **Status:** Pass

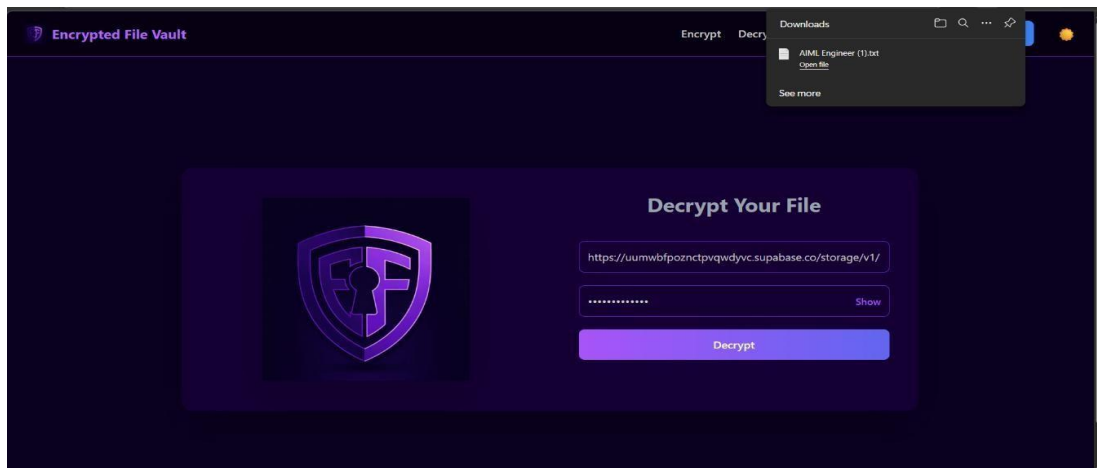


Fig: 6.3.3 Successful File Decryption

Test Case 3: Successful File Decryption

- **Test Input:** Provide a valid encrypted file along with the correct password used during encryption.
- **Expected Outcome:** The system should decrypt the encrypted content and restore the original file data successfully with original filename and file-extension.
- **Actual Result:** The encrypted file was decrypted successfully and the original file content was restored preserving the properties of the file.
- **Status:** Pass

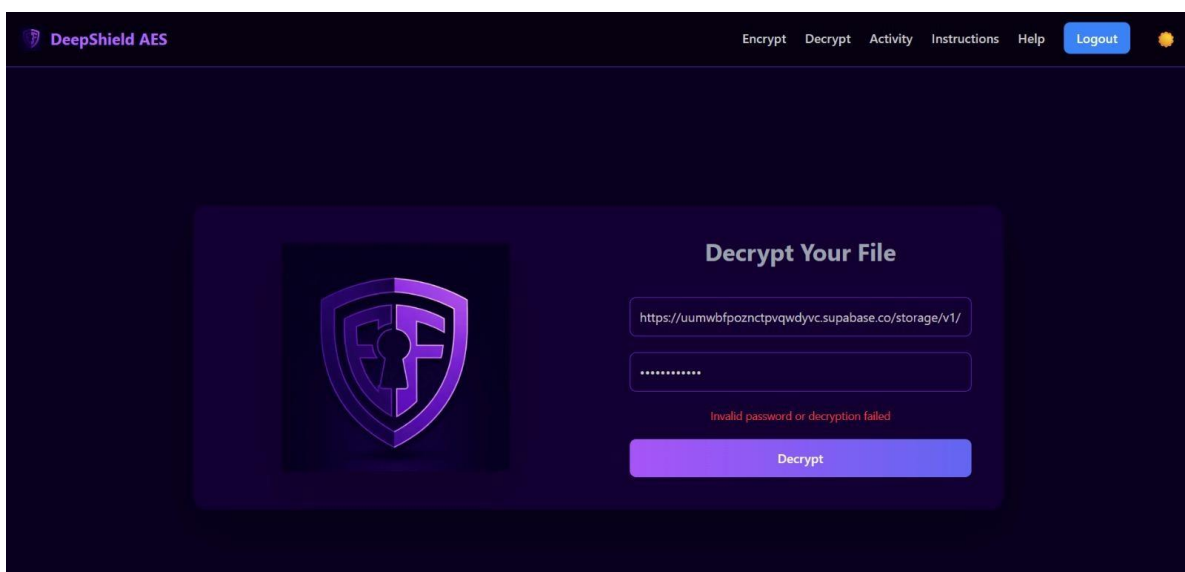


Fig: 6.3.4 Decryption with Incorrect Password

Test Case 4: Decryption with Incorrect Password

- **Test Input:** Attempt to decrypt an encrypted file using an incorrect password.
- **Expected Outcome:** The system should reject the decryption request and display an error message indicating that decryption failed due to an incorrect password or invalid data.
- **Actual Result:** The system returned a message indicating that decryption failed.
- **Status:** Pass

```
if __name__ == "__main__":
    message = b"Cryptography"
    password = "Hi12"

    encrypted = encrypt_data(message, password)
    print("Encrypted (hex):", encrypted['ciphertext'].hex())

    decrypted = decrypt_data(encrypted, password)
    print("Decrypted:", decrypted.decode())

.. ciphertext : b'\x1b\xed\xfa9\xae\xbbt!w\xa7\xa2\x07'
   Encrypted (hex): 1bedf439aebb742157a7a207
   Decrypted: Cryptography

if __name__ == "__main__":
    message = b"Cryptography"
    password = "Hi12"

    encrypted = encrypt_data(message, password)
    print("Encrypted (hex):", encrypted['ciphertext'].hex())

    decrypted = decrypt_data(encrypted, password)
    print("Decrypted:", decrypted.decode())

... ciphertext : b'\x83\x00\x1a\xdb{\tc\x2\xbc\xd6}\x96'
   Encrypted (hex): 83001adb7b0943c2bcd62996
   Decrypted: Cryptography
```

Fig: 6.3.5 Encryption of Same File with Same Password Multiple Times

Test Case 5: Encryption of Same File with Same Password Multiple Times

- **Test Input:** Encrypt the same file twice using the same password.
- **Expected Outcome:** The system should generate different ciphertext outputs each time due to the use of random salt and nonce values.
- **Actual Result:** The encrypted outputs were different for each encryption attempt.
- **Status:** Pass

The systematic testing process confirmed that the DeepShield AES system operates securely and reliably. The encryption mechanism successfully protects file content, and the authentication-based decryption ensures controlled access. The use of AES-GCM further guarantees integrity by preventing unauthorized modifications. All critical test cases passed successfully, demonstrating that the system meets its functional and security requirements.

7. RESULTS OF PROPOSED SYSTEM

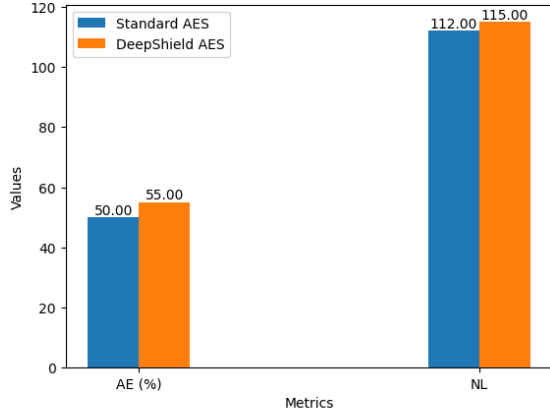
The increased avalanche effect in DeepShield AES indicates stronger diffusion, ensuring that small changes in plaintext or key result in more substantial and unpredictable changes in ciphertext. This enhancement improves resistance against differential cryptanalysis. The improved SAC performance demonstrates that DeepShield AES ensures more balanced and uniform output bit changes, indicating enhanced randomness and robustness against statistical attacks. The increase in BIC suggests that output bits in DeepShield AES behave more independently, making it harder for attackers to exploit correlations between ciphertext bits. Higher non-linearity in DeepShield AES confirms that the modified S-box structure introduces stronger confusion properties, thereby increasing resistance to linear attacks. The unchanged DU value confirms that DeepShield AES preserves the fundamental security guarantees of AES while enhancing other cryptographic properties.

Fixed Plain Text			
Test Case	Cipher Text	Number of Bits Changed	Avalanche Effect (%)
1	174cfa0c5608dbf0ef100667	51	53.12
2	3522237286f4b3d0e72ab216	51	53.12
3	a5f0d63eaa0015e72a26584b	58	60.42
4	25d90260720fdfafe283471c	52	54.17
5	d717e4233fdda3ff350fa492	53	55.21
6	6068feafbe67431da2fb4215	53	55.21
7	d03a268d9a8a2cf22f2c13eb	52	54.17
8	3a641ca2fa8b6936c5bb9fa3	53	55.21
9	735837eb1fa9fd92c7bfdeda	57	59.38
10	a7432a5a7d91c8e2a7b5d962	53	55.21

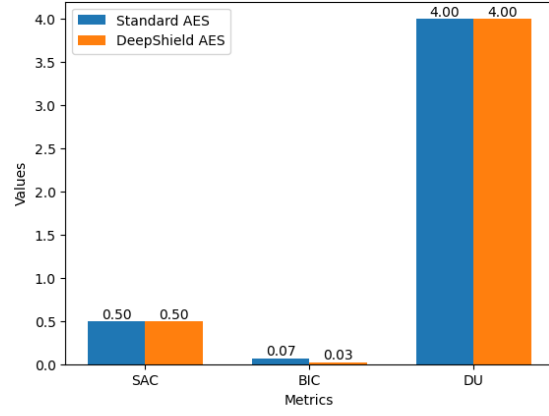
Table 1. Avalanche Effect of DeepShield over fixed plain text and variable keys

Table 1 evaluates the avalanche effect of DeepShield AES using a fixed plaintext while varying the encryption key across 20 test cases. The number of changed bits ranges from 51 to 59, corresponding to avalanche effects between 53.12% and 61.46%.

These results confirm that DeepShield AES maintains key sensitivity, ensuring that even minor changes in the encryption key produce substantial changes in the ciphertext. This behavior is essential for resisting related-key attacks and confirms the robustness of the proposed cipher under practical usage scenarios.



Graph 1: AE and NL Comparison



Graph 2: SAC, BIC and DU Comparison

Graph 1 compares the Avalanche Effect (AE) and Nonlinearity (NL) between Standard AES and the proposed DeepShield AES algorithm. A higher AE value indicates stronger diffusion and better resistance against cryptographic attacks. From the graph, DeepShield AES achieves an AE of 55%, which is higher than Standard AES at 50%, demonstrating improved sensitivity to input changes and enhanced security. Higher NL values indicate better resistance to linear and differential cryptanalysis. DeepShield AES shows a slightly higher NL value of 115 compared to 112 in Standard AES suggesting stronger resistance to advanced attacks.

Overall, Graph 1 indicates that DeepShield AES provides improved security characteristics over Standard AES in terms of diffusion and nonlinearity.

Graph 2 presents a comparison of Strict Avalanche Criterion (SAC), Bit Independence Criterion (BIC) and Diffusion Uniformity (DU) between Standard AES and DeepShield AES. Both algorithms achieve a SAC value of 0.50, indicating optimal and ideal avalanche behaviour. DeepShield AES shows a lower BIC value (0.03) compared to Standard AES (0.07), highlighting improved bit independence and stronger resistance to statistical attacks. Both algorithms achieve the same DU value of 4.00, indicating similar performance in uniform diffusion.

In summary, Graph 2 demonstrates that while both algorithms perform equally well in SAC and DU, DeepShield AES offers better bit independence, making it more secure against correlation-based cryptographic attacks.

8. CONCLUSION

In today's digital environment, the security of sensitive information has become increasingly important due to the growing use of cloud storage systems and online platforms. Protecting confidential files from unauthorized access, data breaches, and cyber threats requires strong cryptographic mechanisms combined with reliable system design. This project, "DeepShield AES – A Dynamic S-Box Based Enhancement for Secure File Encryption" presents a secure and efficient approach for protecting digital files through advanced encryption techniques integrated within a web-based application called "DeepShield".

The proposed system enhances the traditional AES encryption framework by introducing a dynamic chaotic S-Box mechanism, which increases the complexity and unpredictability of the encryption process. Unlike static substitution boxes used in standard AES implementations, the dynamic S-Box in DeepShield AES is generated using chaotic mathematical functions, making the encryption process more resistant to cryptographic attacks such as differential and linear cryptanalysis. Additionally, the system incorporates PBKDF2 with SHA-512 for secure password-based key derivation, which strengthens protection against brute-force, dictionary, and rainbow table attacks.

The implementation of AES-256 in Galois Counter Mode (AES-GCM) further enhances the security of the system by providing both confidentiality and integrity verification. AES- GCM ensures that encrypted data cannot be modified without detection, thereby protecting files from tampering and unauthorized manipulation. The combination of dynamic substitution, secure key derivation, and authenticated encryption creates a multi-layer security architecture capable of safeguarding sensitive data.

To demonstrate the practical applicability of the proposed encryption model, the system was implemented as a modern web application. The platform enables users to securely upload, encrypt, store, and retrieve files through an intuitive interface. The frontend was developed using ReactJS, while backend operations and cryptographic processing were handled through Node.js and Express.js, with MongoDB Atlas managing user and file metadata. Encrypted files are securely stored in cloud storage, ensuring that only encrypted data is maintained outside the application environment.

Extensive system testing confirmed that the proposed solution performs reliably under various scenarios. The system successfully encrypts and decrypts files, prevents unauthorized access through strong authentication mechanisms and detects tampering attempts using AES-GCM integrity checks. The use of random salts and nonces ensures that even identical files encrypted with the same password produce different ciphertext outputs, thereby improving overall security.

In conclusion, the DeepShield AES system provides a secure, scalable and efficient solution for protecting digital files. By combining advanced cryptographic techniques with modern web technologies, the project demonstrates how strong encryption can be effectively integrated into practical applications for secure data storage and management. The DeepShield platform highlights the potential of enhanced AES encryption mechanisms in building reliable systems capable of addressing modern cybersecurity challenges.

9. BIBLIOGRAPHY

1. M. R. Bayesh, D. Das, and M. Ahadullah, "A dual-layer image encryption framework using chaotic AES with dynamic S-boxes and steganographic QR codes," *Journal of Information Security and Applications*, 2026.
2. R. Flores-Carapia *et al.*, "A chaotic digital signature algorithm based on a dynamic substitution box," *Scientific Reports*, vol. 15, 2025.
3. H. Li, Z. Wang, and Y. Zhang, "An optimized chaotic S-box for real-time image encryption scheme based on hyperchaotic maps," *Ain Shams Engineering Journal*, vol. 15, no. 4, 2024.
4. H. Calvo, D. Nakojah, and M. Madani, "FPGA implementation of AES based on optimized dynamic S-box," in *Proc. Beyond Technology Summit on Informatics International Conference (BTS-I2C)*, 2024.
5. X. Hu, X. Zeng, Z. Liu, and G. Yang, "An affine equivalence algorithm for S-boxes based on matrix invariants," 2024.
6. M. Shahbaz Khan, A. Al-Dubai, and J. Ahmad, "A novel chaotic image encryption scheme for secure communication," *IEEE Access*, vol. 11, pp. 112345–112357, 2023.
7. Y. Wang and K. Wong, "A new chaotic cryptographic scheme using dynamic S-boxes," *Chaos, Solitons & Fractals*, vol. 41, no. 4, pp. 1773–1779, 2019.
8. M. Khan and T. Shah, "A novel image encryption technique based on chaotic maps and dynamic substitution boxes," *Signal Processing*, vol. 150, pp. 126–137, 2018.
9. F. Özkaynak and A. B. Özer, "A method for designing strong S-boxes based on chaotic systems," *Nonlinear Dynamics*, vol. 86, no. 2, pp. 1213–1226, 2016.

10. D. S. Abdul Elminaam, H. M. Abdul Kader, and M. M. Hadhoud, "Performance evaluation of symmetric encryption algorithms," *International Journal of Computer Science and Network Security*, vol. 8, no. 12, pp. 280–286, 2010.
11. J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*. Berlin, Germany: Springer-Verlag, 2002.
12. NIST, "Advanced Encryption Standard (AES)," *Federal Information Processing Standards Publication (FIPS) 197*, National Institute of Standards and Technology, 2001.

10. APPENDIX

10.1 Appendix-A

Project Repository Details

Project Title: DeepShield AES: A Dynamic S-Box based Enhancement for Secure File Encryption

Batch: B7

Batch Members:

K. Susmitha Lakshmi	(22B01A0579)
K. Sree Karthika	(22B01A05A0)
K. Harika	(22B01A05A2)
M.Varshini	(22B01A05B6)
M. Valli Gayathri	(22B01A05C1)

Department: Computer Science and Engineering

Institution: Shri Vishnu Engineering College for Women

Guide: Dr. P. Sri Ram Chandra M.Tech, Ph.D, Associate Professor

GitHub Repository Link:

The complete project files including source code, documentation and additional resources are available at the following GitHub repository:

<https://github.com/ValliGayathri/DeepShield>

For quick access, scan the QR code below:



10.1.1 QR Code

Live Website: <https://deep-shield-6ayc.vercel.app/>

For quick access, scan the QR code below:



10.1.2 QR Code

10.2 Appendix-B

10.2.1 Introduction to Python

Python is a high-level, interpreted programming language widely recognized for its simplicity, readability, and versatility. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming, which makes it ideal for both rapid prototyping and full-scale application development. In the context of the DeepShield project, Python serves as the backbone of the backend server, enabling the implementation of secure API endpoints, authentication mechanisms, and file management functionalities. Its extensive standard library and robust third-party ecosystem provide tools for cryptographic operations, web server management, and cloud integration. Python's clear syntax and readability also facilitate maintainability, allowing developers to implement complex logic, such as encryption algorithms and key derivation functions, without compromising on clarity or performance. By leveraging Python, DeepShield achieves a secure, efficient, and scalable backend environment suitable for handling sensitive file encryption and decryption tasks while integrating seamlessly with modern web technologies.

10.2.2 Introduction to Flask

Flask is a lightweight, Python-based web framework designed to simplify the development of web applications while providing flexibility and scalability. Unlike full-stack frameworks, Flask follows a micro-framework philosophy, giving developers control over the application structure and third-party integrations. In DeepShield, Flask powers the server-side operations, handling HTTP requests, routing, and communication with the frontend interface. It manages user authentication, session handling and secure interaction with the cloud storage

system for encrypted files. Flask's modular design allows the backend to be organized into reusable components like authentication middleware, encryption logging and file management modules. Additionally, Flask supports integration with external libraries for database operations, RESTful APIs, and cryptography, enabling the seamless implementation of client-server workflows. By using Flask, DeepShield ensures a lightweight yet secure backend capable of supporting dynamic encryption processes while providing responsive endpoints for client-side applications.

10.2.3 Introduction to HTML

HTML (HyperText Markup Language) is the standard markup language used for creating the structure and content of web pages. It defines elements such as headings, paragraphs, forms, links, and interactive components, forming the foundation of web interfaces. In this project, HTML is utilized to create a structured, user-friendly interface for uploading files, entering passwords, and accessing encrypted content. Forms are designed to accept sensitive user inputs, while links and buttons facilitate secure navigation across the application. HTML provides semantic elements that improve accessibility and compatibility with modern web browsers, ensuring that the DeepShield platform is accessible to a wide range of users. By combining HTML with client-side scripts and styling frameworks, the platform achieves a coherent structure that supports complex workflows, such as client-side encryption, secure file uploads, and dynamic display of encryption/decryption results, while maintaining usability and readability.

10.2.4 Introduction to Tailwind CSS

Tailwind CSS is a modern, utility-first CSS framework that provides pre-defined classes for rapidly designing responsive and consistent user interfaces. Unlike traditional CSS frameworks, Tailwind allows developers to apply styling directly in HTML elements, reducing the need for custom CSS files and promoting a modular approach to design. In this application, Tailwind CSS is employed to create a clean, visually appealing, and responsive interface for both desktop and mobile users. It allows precise control over spacing, typography, color schemes, and interactive states, ensuring that elements like file upload forms, encryption buttons, and notifications are visually consistent and intuitive. Tailwind's responsive utilities facilitate automatic adjustments across different screen sizes, improving usability and user experience. By adopting Tailwind CSS, the system achieves a professional and modern interface while minimizing development overhead and ensuring maintainable styling practices.

10.2.5 Introduction to JavaScript

JavaScript is a high-level, interpreted scripting language primarily used to make web applications interactive and dynamic. It runs directly in the user's browser, enabling real-time execution of functions without relying on server responses. JavaScript plays a critical role in implementing client-side encryption workflows, including PBKDF2 key derivation, dynamic S-Box generation, and AES-GCM encryption of uploaded files. By performing these operations on the client side, sensitive data never leaves the user's device in an unencrypted form, enhancing security and privacy. JavaScript also manages user interactions such as form validation, password visibility toggling, progress indicators, and secure communication with the Flask backend. Its event-driven model and support for asynchronous operations enable smooth handling of large files and responsive UI updates. Using JavaScript, the system ensures a seamless, interactive experience for users while maintaining robust security standards for end-to-end encrypted file handling.